

ГЛАВА 7

КЛАССЫ

В этой главе...

7.1. Определение абстрактных типов данных	332
7.2. Управление доступом и инкапсуляция	349
7.3. Дополнительные средства класса	353
7.4. Область видимости класса	366
7.5. Снова о конструкторах	372
7.6. Статические члены класса	387
Резюме	392
Термины	393

Классы в языке C++ используются для определения собственных типов данных. Определение типов, отражающих концепции решаемых задач, позволяет существенно упростить написание, отладку и модификацию программ.

В этой главе будет продолжено описание классов, начатое в главе 2. Основное внимание здесь уделяется важности абстракции данных, позволяющей отделять реализацию объекта от операций, в которых объект может участвовать. В главе 13 будет описано, как контролировать происходящее при копировании, перемещении, присвоении и удалении объекта, а в главе 14 рассматривается определение собственных операторов.

Фундаментальными идеями, лежащими в основе концепции классов (class), являются *абстракция данных* (data abstraction) и *инкапсуляция* (encapsulation). Абстракция данных — программный подход, полагающийся на разделение *интерфейса* (interface) и *реализации* (implementation). Интерфейс класса состоит из операций, которые пользователь класса может выполнить с его объектом. Реализация включает переменные-члены класса, тела функций, составляющих интерфейс, а также любые функции, которые нужны для определения класса, но не предназначены для общего использования.

Инкапсуляция обеспечивает разделение интерфейса и реализации класса. Инкапсулируемый класс скрывает свою реализацию от пользователей, которые могут использовать интерфейс, но не имеют доступа к реализации класса.

Класс, использующий абстракцию данных и инкапсуляцию, называют *абстрактным типом данных* (abstract data type). Внутренняя реализация абстрактного типа данных заботит только его разработчика. Программисты, которые используют этот класс, не обязаны ничего знать о том, как внутренне работает этот тип. Они могут рассматривать его как *абстракцию*.

7.1. Определение абстрактных типов данных

Класс `Sales_item`, использованный в главе 1, является абстрактным типом данных. При использовании объекта класса `Sales_item` задействовался его интерфейс (т.е. операции, описанные в разделе 1.5.1 (стр. 47)). Мы не имели доступа к переменным-членам, хранящимся в объекте класса `Sales_item`. На самом деле нам даже не было известно, какие переменные-члены имеет этот класс.

Наш класс `Sales_data` (см. раздел 2.6.1, стр. 112) не был абстрактным типом данных. Он позволяет пользователям обращаться к его переменным-членам и вынуждает пользователей писать собственные операции. Чтобы сделать класс `Sales_data` абстрактным типом, необходимо определить операции, доступные для его пользователей. Как только класс `Sales_data` определит собственные операции, мы сможем инкапсулировать (т.е. скрыть) его переменные-члены.

7.1.1. Разработка класса `Sales_data`

В конечном счете хочется, чтобы класс `Sales_data` поддерживал тот же набор операций, что и класс `Sales_item`. У класса `Sales_item` была одна *функция-член* (member function) (см. раздел 1.5.2, стр. 51) по имени `isbn`, а также поддерживались операторы `+`, `=`, `<<` и `>>`.

Определение собственных операторов рассматривается в главе 14, а пока определим обычные (именованные) функции для этих операций. По причинам, рассматриваемым в разделе 7.1.5 (стр. 704), функции, осуществляющие сложение и операции ввода-вывода, не будут членами класса `Sales_data`. Мы определим эти функции как обычные. Функция, выполняющая составное присвоение, будет членом класса, и по причинам, рассматриваемым в разделе 7.1.5 (стр. 347), наш класс не должен определять присвоение.

Таким образом, интерфейс класса `Sales_data` состоит из следующих операций.

- Функция-член `isbn()`, возвращающая ISBN объекта.
- Функция-член `combine()`, добавляющая один объект класса `Sales_data` к другому.
- Функция `add()`, суммирующая два объекта класса `Sales_data`.

- Функция `read()`, считывающая данные из потока `istream` в объект класса `Sales_data`.
- Функция `print()`, выводящая значение объекта класса `Sales_data` в поток `ostream`.

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. РАЗЛИЧИЕ В РОЛЯХ ПРОГРАММИСТОВ

Пользователями (*user*) программисты обычно называют людей, использующих их приложения. Аналогично разработчик класса реализует его для *пользователей* класса. В данном случае пользователем является другой программист, а не конечный пользователь приложения.

Когда упоминается *пользователь*, имеющееся в виду лицо определяет контекст употребления термина. Если речь идет о *пользовательском коде* или *пользователе* класса `Sales_data`, то подразумевается программист, который использует класс. Если речь идет о *пользователе* приложения книжного магазина, то подразумевается менеджер склада, использующий приложение.



Говоря о *пользователях*, программисты C++, как правило, имеют в виду как пользователей приложения, так и пользователей класса.

В простых приложениях пользователь класса вполне может быть и его разработчиком. Но даже в таких случаях имеет смысл различать роли. Разрабатывая интерфейс класса, следует думать о том, чтобы его было проще использовать. При использовании класса не нужно думать, как именно он работает.

Авторы хороших приложений добиваются успеха потому, что понимают и реализуют потребности пользователей. Точно так же хорошие разработчики класса обращают пристальное внимание на потребности программистов, которые будут использовать их класс. У хорошо разработанного класса удобный, интуитивно понятный интерфейс, а его реализация достаточно эффективна для решения задач пользователя.

Использование пересмотренного класса `Sales_data`

Прежде чем думать о реализации нашего класса, обдумаем то, как можно использовать функции его интерфейса. В качестве примера использования этих функций напомним новую версию программы книжного магазина из раздела 1.6 (стр. 52), работающую с объектами класса `Sales_data`, а не `Sales_item`:

```
Sales_data total;           // переменная для хранения текущей суммы
if (read(cin, total)) {    // прочить первую транзакцию
    Sales_data trans;       // переменная для хранения данных следующей
                             // транзакции
```

```

while(read(cin, trans)) {    // читать остальные транзакции
    if (total.isbn() == trans.isbn()) // проверить isbn
        total.combine(trans); // обновить текущую сумму
    else {
        print(cout, total) << endl; // отобразить результаты
        total = trans; // обработать следующую книгу
    }
}
print(cout, total) << endl; // отобразить последнюю транзакцию
} else {                    // ввода нет
    cerr << "No data?!" << endl; // уведомить пользователя
}

```

Сначала определяется объект класса `Sales_data` для хранения текущей суммы. В условии оператора `if` происходит вызов функции `read()` для чтения в переменную `total` первой транзакции. Это условие работает, как и другие написанные ранее циклы с использованием оператора `>>`. Как и оператор `>>`, наша функция `read()` будет возвращать свой потоковый параметр, который и проверяет условие (см. раздел 4.11.2, стр. 222). Если функция `read()` потерпит неудачу, сработает часть `else`, выводящая сообщение об ошибке.

Если данные прочитаны успешно, определяем переменную `trans` для хранения всех транзакций. Условие цикла `while` также проверяет поток, возвращенный функцией `read()`. Пока операции ввода в функции `read()` успешны, условие выполняется и обрабатывается следующая транзакция.

В цикле `while` происходит вызов функции-члена `isbn()` объектов `total` и `trans`, возвращающей их ISBN. Если объекты `total` и `trans` относятся к той же книге, происходит вызов функции `combine()`, добавляющей компоненты объекта `trans` к текущей сумме, хранящейся в объекте `total`. Если объект `trans` представляет новую книгу, происходит вызов функции `print()`, выводящей итог по предыдущей книге. Поскольку функция `print()` возвращает ссылку на свой потоковый параметр, ее результат можно использовать как левый операнд оператора `<<`. Это сделано для того, чтобы вывести символ новой строки после результата, созданного функцией `print()`. Затем объект `trans` присваивается объекту `total`, начиная таким образом обработку записи следующей книги в файле.

По исчерпанию ввода следует не забыть вывести данные последней транзакции. Для этого после цикла `while` используется еще один вызов функции `print()`.

Упражнения раздела 7.1.1

Упражнение 7.1. Напишите версию программы обработки транзакций из раздела 1.6 (стр. 52) с использованием класса `Sales_data`, созданного для упражнений в разделе 2.6.1 (стр. 112).



7.1.2. Определение пересмотренного класса `Sales_data`

У пересмотренного класса будут те же переменные-члены, что и у версии, определенной в разделе 2.6.1 (стр. 112): член типа `string` по имени `bookNo`, представляющий ISBN, член типа `unsigned` по имени `units_sold`, представляющий количество проданных экземпляров книги, и член типа `double` по имени `revenue`, представляющий общий доход от этих продаж.

Как уже упоминалось, у класса будут также две функции-члена, `combine()` и `isbn()`. Кроме того, предоставим классу `Sales_data` другую функцию-член, чтобы возвращать среднюю цену, по которой были проданы книги. Эта функция, назовем ее `avg_price()`, не предназначена для общего использования. Она будет частью реализации, а не интерфейса.

Функции-члены определяют (см. раздел 6.1, стр. 267) и объявляют (см. раздел 6.1.2, стр. 273) как обычные функции. Функции-члены *должны быть* объявлены в классе, но определены они *могут* быть непосредственно в классе или вне тела класса. Функции, не являющиеся членами класса, но являющиеся частью интерфейса, как функции `add()`, `read()` и `print()`, объявляются и определяются вне класса.

С учетом вышеизложенного напомним пересмотренную версию класса `Sales_data`:

```
struct Sales_data {
    // новые члены: операции с объектами класса Sales_data
    std::string isbn() const { return bookNo; }
    Sales_data& combine(const Sales_data&);
    double avg_price() const;
    // те же переменные-члены, что и в р 2.6.1 (стр. 112)
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
// функции интерфейса класса Sales_data, не являющиеся его членами
Sales_data add(const Sales_data&, const Sales_data&);
std::ostream &print(std::ostream&, const Sales_data&);
std::istream &read(std::istream&, Sales_data&);
```



Функции, определенные в классе, неявно являются встраиваемыми (см. раздел 6.5.2, стр. 312).

Определение функций-членов

Хотя каждый член класса должен быть объявлен в самом классе, тело функции-члена можно определить либо в, либо вне тела класса. Функция `isbn()` определяется в классе `Sales_data`, а функции `combine()` и `avg_price()` вне его.

Сначала рассмотрим функцию `isbn()`, возвращающую строку и имеющую пустой список параметров:

```
std::string isbn() const { return bookNo; }
```

Как и у любой функции, тело функции-члена является блоком. В данном случае блок содержит один оператор `return`, возвращающий значение переменной-члена `bookNo` объекта класса `Sales_data`. Интересно, как эта функция получает объект, член `bookNo` которого следует выбрать?

Указатель `this`

Давайте снова рассмотрим вызов функции-члена `isbn()`:

```
total.isbn()
```

Здесь для вызова функции-члена `isbn()` объекта `total` используется точечный оператор (см. раздел 4.6, стр. 207).

За одним исключением, рассматриваемым в разделе 7.6 (стр. 387), вызов функции-члена осуществляется от имени объекта. Когда функция `isbn()` обращается к члену класса `Sales_data` (например, `bookNo`), она неявно обращается к членам того объекта, из которого была вызвана. В этом вызове функции `isbn()`, когда она возвращает значение члена `bookNo`, речь идет о члене `total.bookNo`.

Функция-член способна обратиться к тому объекту, из которого она была вызвана, благодаря дополнительному неявному параметру `this`. Когда происходит вызов функции-члена, указатель `this` инициализируется адресом объекта, из которого была вызвана функция. Рассмотрим следующий вызов:

```
total.isbn()
```

Здесь компилятор присваивает адрес объекта `total` указателю `this` и неявно передает его как параметр функции `isbn()`. Компилятор как бы переписывает этот вызов так:

```
// псевдокод, в который преобразуется вызов функции-члена  
Sales_data::isbn(&total)
```

Этот код вызывает функцию-член `isbn()` класса `Sales_data`, передав адрес объекта `total`.

В функции-члене можно обратиться непосредственно к членам объекта, из которого она была вызвана. Для использования членов объекта, на который указывает указатель `this`, можно не использовать оператор доступа к члену. Любое непосредственное использование члена класса подразумевает использование указателя `this`. Таким образом, когда функция `isbn()` использует переменную `bookNo`, она неявно использует член объекта, на который указывает указатель `this`. Это аналогично синтаксису `this->bookNo`.

Параметр `this` определяется неявно и автоматически. Кроме того, определить параметр или переменную по имени `this` самому нельзя, но в теле функции-члена его использовать можно. Вполне допустимо, хоть и ненужно, определить функцию `isbn()` так:

```
std::string isbn() const { return this->bookNo; }
```

Поскольку указатель `this` всегда предназначен для обращения к “этому” объекту, он является константным (см. раздел 2.4.2, стр. 99). Нельзя изменить адрес, хранящийся в указателе `this`.

Константные функции-члены

Еще одним важным моментом функции-члена `isbn()` является ключевое слово `const`, расположенное за списком параметров. Оно применяется для модификации типа неявного указателя `this`.

По умолчанию указатель `this` имеет тип константного указателя на не-константную версию типа класса. Например, типом по умолчанию указателя `this` в функции-члене `Sales_data` является `Sales_data *``const`. Хотя указатель `this` и неявен, он подчиняется обычным правилам инициализации, согласно которым (по умолчанию) нельзя связать указатель `this` с константным объектом (см. раздел 2.4.2, стр. 99). Следствием этого факта, в свою очередь, является невозможность вызвать обычную функцию-член для константного объекта.

Если бы функция `isbn()` была обычной и если бы указатель `this` был обычным параметром типа указателя, то мы объявили бы его как `const Sales_data *``const`. В конце концов, тело функции `isbn()` не изменяет объект, на который указывает указатель `this`; таким образом, эта функция стала бы гибче, если бы указатель `this` был указателем на константу (см. раздел 6.2.3, стр. 281).

Однако указатель `this` неявный и не присутствует в списке параметров, поэтому нет места, где можно было бы указать, что он должен быть указателем на константу. Язык решает эту проблему, позволяя разместить ключевое слово `const` после списка параметров функции-члена. Это означает, что указатель `this` является указателем на константу. Функции-члены, использующие ключевое слово `const` таким образом, являются *константными функциями-членами* (`const member function`).

Тело функции `isbn()` можно считать написанным так:

```
// псевдокод, иллюстрирующий использование неявного указателя
// этот код недопустим: нельзя самому явно определить этот указатель
// обратите внимание, что это указатель на константу, поскольку isbn()
// является константным членом класса
std::string Sales_data::isbn(const Sales_data *const this)
{ return this->isbn; }
```

Тот факт, что `this` является указателем на константу, означает, что константные функции-члены не могут изменить объект, для которого они вызваны. Таким образом, функция `isbn()` может читать значения переменных-членов объектов, для которых она вызывается, но не изменять их.



Константные объекты, ссылки и указатели на константные объекты могут вызывать только константные функции-члены.

Область видимости класса и функции-члены

Помните, что класс сам является областью видимости (см. раздел 2.6.1, стр. 112). Определения функций-членов класса находятся в области видимости самого класса. Следовательно, использованное функцией `isbn()` имя `bookNo` относится к переменной-члену, определенной в классе `Sales_data`.

Следует заметить, что функция `isbn()` может использовать имя `bookNo`, несмотря на то, что оно определено *после* функции `isbn()`. Как будет описано в разделе 7.4.1 (стр. 367), компилятор обрабатывает классы в два этапа — сначала объявления членов класса, затем тела функций-членов, если таковые вообще имеются. Таким образом, тела функций-членов могут использовать другие члены своих классов, независимо от того, где именно в классе они определены.

Определение функции-члена вне класса

Подобно любой другой функции, при определении функции-члена вне тела класса ее определение должно соответствовать объявлению. Таким образом, тип возвращаемого значения, список параметров и имя должны совпадать с объявлением в теле класса. Если член класса был объявлен как константная функция, то в определении после списка параметров также должно присутствовать ключевое слово `const`. Имя функции-члена, определенное вне класса, должно включить имя класса, которому она принадлежит:

```
double Sales_data::avg_price() const {
    if (units_sold)
        return revenue/units_sold;
    else
        return 0;
}
```

Имя функции, `Sales_data::avg_price()`, использует оператор области видимости (см. раздел 1.2, стр. 33), чтобы указать, что определяемая функция по имени `avg_price` объявлена в пределах класса `Sales_data`. Как только компилятор увидит имя функции, остальная часть кода интерпретируется как относящаяся к области видимости класса. Таким образом, когда функция `avg_price()` обращается к переменным `revenue` и `units_sold`, она неявно имеет в виду члены класса `Sales_data`.

Определение функции, возвращающей указатель `this` на объект

Функция `combine()` должна действовать как составной оператор присвоения `+=`. Объект, для которого вызвана эта функция, представляет собой левый операнд присвоения. Правый операнд передается как аргумент явно:

```
Sales_data& Sales_data::combine(const Sales_data &rhs)
{
    units_sold += rhs.units_sold; // добавить члены объекта rhs
    revenue += rhs.revenue;       // к членам объекта 'this'
    return *this; // вернуть объект, для которого вызвана функция
}
```

Когда наша программа обработки транзакций осуществляет вызов

```
total.combine(trans); // обновить текущую сумму
```

адрес объекта `total` находится в неявном параметре `this`, а объект `trans` связан с параметром `rhs`. Таким образом, при вызове функции `combine()` выполняется следующий оператор:

```
units_sold += rhs.units_sold; // добавить члены объекта rhs
```

В результате произойдет сложение переменных `total.units_sold` и `trans.units_sold`, а сумма должна сохраниться снова в переменной `total.units_sold`.

Самым интересным в этой функции является тип возвращаемого значения и оператор `return`. Обычно при определении функции, работающей как стандартный оператор, она должна подражать поведению этого оператора. Стандартные операторы присвоения возвращают свой левый операнд как l-значение (см. раздел 144, стр. 201). Чтобы вернуть l-значение, наша функция `combine()` должна вернуть ссылку (см. раздел 6.3.2, стр. 296). Поскольку левый операнд — объект класса `Sales_data`, тип возвращаемого значения — `Sales_data&`.

Как уже упоминалось, для доступа к члену объекта, функция-член которого выполняется, необязательно использовать неявный указатель `this`. Однако для доступа к объекту в целом указатель `this` действительно нужен:

```
return *this; // вернуть объект, для которого вызвана функция
```

Здесь оператор `return` обращается к значению указателя `this`, чтобы получить объект, функция которого выполняется. Таким образом, для этого вызова возвращается ссылка на объект `total`.

Упражнения раздела 7.1.2

Упражнение 7.2. Добавьте функции-члены `combine()` и `isbn()` в класс `Sales_data`, который был написан для упражнений из раздела 2.6.2 (стр. 116).

Упражнение 7.3. Пересмотрите свою программу обработки транзакций из раздела 7.1.1 (стр. 333) так, чтобы использовать эти функции-члены.

Упражнение 7.4. Напишите класс по имени `Person`, представляющий имя и адрес человека. Используйте для содержания каждого из этих членов тип `string`. В последующих упражнениях в этот класс будут добавлены новые средства.

Упражнение 7.5. Снабдите класс `Person` операциями возвращения имени и адреса. Должны ли эти функции быть константами? Объясните свой выбор.



7.1.3. Определение функций, не являющихся членом класса, но связанных с ним

Авторы классов нередко определяют такие вспомогательные функции, как наши функции `add()`, `read()` и `print()`. Хотя определяемые ими операции концептуально являются частью интерфейса класса, частью самого класса они не являются.

Мы определяем функции, не являющиеся членом класса, как любую другую функцию, т.е. ее объявление обычно отделено от определения (см. раздел 6.1.2, стр. 273). Функции, концептуально являющиеся частью класса, но не определенные в нем, как правило, объявляются (но не определяются) в том же заголовке, что и сам класс. Таким образом, чтобы использовать любую часть интерфейса, пользователю достаточно включить только один файл.



Обычно функция, не являющаяся членом класса, но из состава его интерфейса объявляется в том же заголовке, что и сам класс.

Определение функций `read()` и `print()`

Функции `read()` и `print()` выполняют ту же задачу, что и код в разделе 2.6.2 (стр. 115), поэтому и не удивительно, что тела этих функций очень похожи на код, представленный там:

```
// введенные транзакции содержат ISBN, количество проданных книг и
// цену книги
istream &read(istream &is, Sales_data &item)
{
    double price = 0;
    is >> item.bookNo >> item.units_sold >> price;
    item.revenue = price * item.units_sold;
    return is;
}

ostream &print(ostream &os, const Sales_data &item)
{
    os << item.isbn() << " " << item.units_sold << " "
```

```

        << item.revenue << " " << item.avg_price();
    return os;
}

```

Функция `read()` читает данные из предоставленного потока в заданный объект. Функция `print()` выводит содержимое предоставленного объекта в заданный поток.

В этих функциях, однако, следует обратить внимание на два момента. Во-первых, обе функции получают ссылки на соответствующие объекты классов ввода и вывода. Классы ввода и вывода — это типы, не допускающие копирования, поэтому их передача возможна только по ссылке (см. раздел 6.2.2, стр. 277). Кроме того, чтение и запись в поток изменяют его, поэтому обе функции получают обычные ссылки, а не ссылки на константы.

Второй заслуживающий внимания момент: функция `print()` не выводит новую строку. Обычно функции вывода осуществляют минимальное форматирование. Таким образом, пользовательский код может сам решить, нужна ли новая строка.

Определение функции `add()`

Функция `add()` получает два объекта класса `Sales_data` и возвращает новый объект класса `Sales_data`, представляющий их сумму:

```

Sales_data add(const Sales_data &lhs, const Sales_data &rhs)
{
    Sales_data sum = lhs; // копирование переменных-членов из lhs в sum
    sum.combine(rhs);     // добавить переменные-члены rhs в sum
    return sum;
}

```

В теле функции определяется новый объект класса `Sales_data` по имени `sum`, предназначенный для хранения суммы двух транзакций. Инициализируем объект `sum` копией объекта `lhs`. По умолчанию копирование объекта класса подразумевает копирование и членов этого объекта. После копирования у членов `bookNo`, `units_sold` и `revenue` объекта `sum` будут те же значения, что и у таковых объекта `lhs`. Затем происходит вызов функции `combine()`, суммирующей значения переменных-членов `units_sold` и `revenue` объектов `rhs` и `sum` в последний. По завершении возвращается копия объекта `sum`.

Упражнения раздела 7.1.3

Упражнение 7.6. Определите собственные версии функций `add()`, `read()` и `print()`.

Упражнение 7.7. Перепишите программу обработки транзакций, написанной для упражнений в разделе 7.1.2 (стр. 339), так, чтобы использовать эти новые функции.

Упражнение 7.8. Почему функция `read()` определяет свой параметр типа `Sales_data` как простую ссылку, а функция `print()` — как ссылку на константу?

Упражнение 7.9. Добавьте в код, написанный для упражнений в разделе 7.1.2 (стр. 339), операции чтения и вывода объектов класса `Person`.

Упражнение 7.10. Что делает условие в следующем операторе `if`?

```
if (read(read(cin, data1), data2))
```

7.1.4. Конструкторы

Каждый класс определяет, как могут быть инициализированы объекты его типа. Класс контролирует инициализацию объекта за счет определения одной или нескольких специальных функций-членов, известных как *конструкторы* (constructor). Задача конструктора — инициализировать переменные-члены объекта класса. Конструктор выполняется каждый раз, когда создается объект класса.

В этом разделе рассматриваются основы определения конструкторов. Конструкторы — удивительно сложная тема. На самом деле мы сможем больше сказать о конструкторах в разделах 7.5 (стр. 372), 15.7 (стр. 784) и 18.1.3 (стр. 973), а также в главе 13.

Имя конструкторов совпадает с именем класса. В отличие от других функций, у конструкторов нет типа возвращаемого значения. Как и другие функции, конструкторы имеют (возможно пустой) список параметров и (возможно пустое) тело. У класса может быть несколько конструкторов. Подобно любой другой перегруженной функции (см. раздел 6.4, стр. 302), конструкторы должны отличаться друг от друга количеством или типами своих параметров.

В отличие от других функций-членов, конструкторы не могут быть объявлены константами (см. раздел 7.1.2, стр. 337). При создании константного объекта типа класса его константность не проявится, пока конструктор не закончит инициализацию объекта. Таким образом, конструкторы способны осуществлять запись в константный объект во время его создания.

Синтезируемый стандартный конструктор

Хотя в нашем классе `Sales_data` не определено конструкторов, использующие его программы компилировались и выполнялись правильно. Например, программа на стр. 333 определяла два объекта класса `Sales_data`:

```
Sales_data total;           // переменная для хранения текущей суммы
Sales_data trans;          // переменная для хранения данных следующей
                           // транзакции
```

Естественно, возникает вопрос: как инициализируются объекты `total` и `trans`?

Настолько известно, инициализатор для этих объектов не предоставлялся, поэтому они инициализируются значением по умолчанию (см. раздел 2.2.1, стр. 77). Классы сами контролируют инициализацию по умолчанию, определяя специальный конструктор, известный как *стандартный конструктор* (default constructor). Стандартным считается конструктор, не получающий никаких аргументов.

Как будет продемонстрировано, стандартный конструктор является особым во многом, например, если класс не определяет конструкторы *явно*, компилятор сам определит стандартный конструктор *неявно*.

Созданный компилятором конструктор известен как *синтезируемый стандартный конструктор* (synthesized default constructor). У большинства классов этот синтезируемый конструктор инициализирует каждую переменную-член класса следующим образом:

- Если есть внутриклассовый инициализатор (см. раздел 2.6.1, стр. 113), он и используется для инициализации члена класса.
- В противном случае член класса инициализируется значением по умолчанию (см. раздел 2.2.1, стр. 77).

Поскольку класс `Sales_data` предоставляет инициализаторы для переменных `units_sold` и `revenue`, синтезируемый стандартный конструктор использует данные значения для инициализации этих членов. Переменная `bookNo` инициализируется значением по умолчанию, т.е. пустой строкой.

Некоторые классы не могут полагаться на синтезируемый стандартный конструктор

Только довольно простые классы, такие как текущий класс `Sales_data`, могут полагаться на синтезируемый стандартный конструктор. Как правило, собственный стандартный конструктор для класса определяют потому, что компилятор создает его, *только если для класса не определено никаких других конструкторов*. Если определен хоть один конструктор, то у класса не будет стандартного конструктора, если не определить его самостоятельно. Основание для этого правила таково: если класс требует контроля инициализации объекта в одном случае, то он, вероятно, потребует его во всех случаях.



Компилятор создает стандартный конструктор автоматически, только если в классе *не объявлено* никаких конструкторов.

Вторая причина для определения стандартного конструктора в том, что у некоторых классов синтезируемый стандартный конструктор работает неправильно. Помните, что определенные в блоке объекты встроенного или составного типа (такого как массивы и указатели) без инициализации имеют не-

определенное значение (см. раздел 2.2.1, стр. 77). Это же относится к не инициализированным членам встроенного типа. Поэтому классы, у которых есть члены встроенного или составного типа, должны либо инициализировать их в классе, либо определять собственную версию стандартного конструктора. В противном случае пользователи могли бы создать объекты с членами, значения которых не определены.



Классы, члены которых имеют встроенный или составной тип, могут полагаться на синтезируемый стандартный конструктор, *только если* у всех таких членов есть внутриклассовые инициализаторы.

Третья причина определения некоторыми классами собственного стандартного конструктора в том, что иногда компилятор не способен создать его. Например, если у класса есть член типа класса и у этого класса нет стандартного конструктора, то компилятор не сможет инициализировать этот член. Для таких классов следует определить собственную версию стандартного конструктора. В противном случае у класса не будет пригодного для использования стандартного конструктора. Дополнительные обстоятельства, препятствующие компилятору создать соответствующий стандартный конструктор, приведены в разделе 13.1.6 (стр. 644).

Определение конструкторов класса `Sales_data`

Определим для нашего класса `Sales_data` четыре конструктора со следующими параметрами:

- Типа `istream&`, для чтения транзакции.
- Типа `const string&` для ISBN; типа `unsigned` для количества проданных книг; типа `double` для цены проданной книги.
- Типа `const string&` для ISBN. Для других членов этот конструктор будет использовать значения по умолчанию.
- Без параметров (т.е. стандартный конструктор). Этот конструктор придется определить, поскольку определены другие конструкторы.

Добавим эти члены в класс так:

```
struct Sales_data {
    // добавленные конструкторы
    Sales_data() = default;
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(std::istream &);
    // другие члены, как прежде
    std::string isbn() const { return bookNo; }
    Sales_data& combine(const Sales_data&);
    double avg_price() const;
```

```
std::string bookNo;
unsigned units_sold = 0;
double revenue = 0.0;
};
```

Что значит = default

Начнем с объяснения стандартного конструктора:

```
Sales_data() = default;
```

В первую очередь обратите внимание на то, что это определение стандартного конструктора, поскольку он не получает никаких аргументов. Мы определяем этот конструктор *только потому*, что хотим предоставить другие конструкторы, но и стандартный конструктор тоже нужен. Этот конструктор должен делать то же, что и синтезируемая версия.



По новому стандарту, если необходимо стандартное поведение, можно попросить компилятор создать конструктор автоматически, указав после списка параметров часть `= default`. Синтаксис `= default` может присутствовать как в объявлении в теле класса, так и в определении вне его. Подобно любой другой функции, если часть `= default` присутствует в теле класса, стандартный конструктор будет встраиваемым; если она присутствует в определении вне класса, то по умолчанию этот член не будет встраиваемым.



Стандартный конструктор работает в классе `Sales_data` только потому, что предоставлены инициализаторы для переменных-членов встроеного типа. Если ваш компилятор не поддерживает внутри-классовые инициализаторы, для инициализации каждого члена класса стандартный конструктор должен использовать список инициализации конструктора (описанный непосредственно ниже).

Список инициализации конструктора

Теперь рассмотрим два других конструктора, которые были определены в классе:

```
Sales_data(const std::string &s): bookNo(s) { }
Sales_data(const std::string &s, unsigned n, double p):
    bookNo(s), units_sold(n), revenue(p*n) { }
```

Новой частью этих определений являются двоеточие и код между ним и фигурными скобками, обозначающими пустые тела функций. Эта новая часть — *список инициализации конструктора* (constructor initializer list), определяющий исходные значения для одной или нескольких переменных-членов создаваемого объекта. Инициализатор конструктора — это список имен переменных-членов класса, каждое из которых сопровождается исходным зна-

чением в круглых (или фигурных) скобках. Если инициализаций несколько, они отделяются запятыми.

Конструктор с тремя параметрами использует первые два параметра для инициализации переменных-членов `bookNo` и `units_sold`. Инициализатор для переменной `revenue` вычисляется при умножении количества проданных книг на их цену.

Конструктор с одним параметром типа `string` использует ее для инициализации переменной-члена `bookNo`, но переменные `units_sold` и `revenue` не инициализируются явно. Когда член класса отсутствует в списке инициализации конструктора, он инициализируется неявно, с использованием того же процесса, что и у синтезируемого стандартного конструктора. В данном случае эти члены инициализируются внутриклассовыми инициализаторами. Таким образом, получающий строку конструктор эквивалентен следующему.

```
// то же поведение, что и у исходного конструктора выше
Sales_data(const std::string &s):
    bookNo(s), units_sold(0), revenue(0){ }
```

Обычно для конструктора лучше использовать внутриклассовый инициализатор, если он есть и присваивает члену класса правильное значение. С другой стороны, если ваш компилятор еще не поддерживает внутриклассовые инициализаторы, то каждый конструктор должен явно инициализировать каждый член встроенного типа.



Конструкторы не должны переопределять внутриклассовые инициализаторы, кроме как при использовании иного исходного значения. Если вы не можете использовать внутриклассовые инициализаторы, каждый конструктор должен явно инициализировать каждый член встроенного типа.

Следует заметить, что у обоих этих конструкторов тела пусты. Единственное, что должны сделать эти конструкторы, — присвоить значения переменным-членам. Если ничего другого делать не нужно, то тело функции пусто.

Определение конструктора вне тела класса

В отличие от наших других конструкторов, конструктору, получающему поток `istream`, действительно есть что делать. В своем теле этот конструктор вызывает функцию `read()`, чтобы присвоить переменным-членам новые значения:

```
Sales_data::Sales_data(std::istream &is)
{
    read(is, *this); // read читает транзакцию из is в текущий объект
}
```

У конструкторов нет типа возвращаемого значения, поэтому определение начинается с имени функции. Подобно любой другой функции-члену, при определении конструктора за пределами тела класса необходимо указать класс, которому принадлежит конструктор. Таким образом, синтаксис `Sales_data::Sales_data` указывает, что мы определяем член класса `Sales_data` по имени `Sales_data`. Этот член класса является конструктором, поскольку его имя совпадает с именем класса.

В этом конструкторе нет списка инициализации конструктора, хотя с технической точки зрения было бы правильней сказать, что список инициализации конструктора пуст. Даже при том, что список инициализации конструктора пуст, члены этого объекта инициализируются прежде, чем выполняется тело конструктора.

Члены, отсутствующие в списке инициализации конструктора, инициализируются соответствующим внутриклассовым инициализатором (если он есть) или значением по умолчанию. Для класса `Sales_data` это означает, что при запуске тела функции на выполнение переменная `bookNo` будет содержать пустую строку, а переменные `units_sold` и `revenue` — значение 0.

Чтобы стало понятней, напомним, что второй параметр функции `read()` является ссылкой на объект класса `Sales_data`. В разделе 7.1.2 (стр. 339) мы обращали внимание на то, что указатель `this` используется для доступа к объекту в целом, а не к его отдельному члену. В данном случае для передачи “этого” объекта в качестве аргумента функции `read()` используется синтаксис `*this`.

Упражнения раздела 7.1.4

Упражнение 7.11. Добавьте в класс `Sales_data` конструкторы и напишите программу, использующую каждый из них.

Упражнение 7.12. Переместите определение конструктора `Sales_data()`, получающего объект `istream`, в тело класса `Sales_data`.

Упражнение 7.13. Перепишите программу со стр. 333 так, чтобы использовать конструктор с параметром `istream`.

Упражнение 7.14. Напишите версию стандартного конструктора, явно инициализирующую переменные-члены значениями, предоставленными внутриклассовыми инициализаторами.

Упражнение 7.15. Добавьте соответствующие конструкторы в класс `Person`.



7.1.5. Копирование, присвоение и удаление

Кроме определения способа инициализации своих объектов, классы контролируют также то, что происходит при копировании, присвоении и удалении объектов класса. Объекты копируются во многих случаях: при инициали-

зации переменной, при передаче или возвращении объекта по значению (см. раздел 6.2.1, стр. 277 и раздел 6.3.2, стр. 294). Объекты присваиваются при использовании оператора присвоения (см. раздел 4.4, стр. 201). Объекты удаляются, когда они прекращают существование, например, при выходе локального объекта из блока, в котором он был создан (см. раздел 6.1.1, стр. 271). Объекты, хранимые в векторе (или массиве), удаляются при удалении вектора (или массива).

Если мы не определим эти операции, компилятор создаст их сам. Обычно создаваемые компилятором версии выполняются, копируя, присваивая или удаляя каждую переменную-член объекта. Например, когда в приложении книжного магазина (см. раздел 7.1.1, стр. 333) компилятор выполняет следующее присвоение:

```
total = trans; // обработать следующую книгу
```

оно выполняется, как будто было написано так:

```
// присвоение по умолчанию для Sales_data эквивалентно следующему:  
total.bookNo = trans.bookNo;  
total.units_sold = trans.units_sold;  
total.revenue = trans.revenue;
```

Более подробная информация об определении собственных версий этих операторов приведена в главе 13.



Некоторые классы не могут полагаться на синтезируемые версии

Хотя компилятор и создает сам операторы копирования, присвоения и удаления, важно понимать, что у некоторых классов их стандартные версии ведут себя неправильно. В частности, синтезируемые версии вряд ли будут правильно работать с классами, которые резервируют ресурсы, располагающиеся вне самих объектов класса. Пример резервирования и управления динамической памятью приведен в главе 12. Как будет продемонстрировано в разделе 13.6 (стр. 640), классы, которые управляют динамической памятью, вообще не могут полагаться на синтезируемые версии этих операций.

Однако следует заметить, что большинство классов, нуждающихся в динамической памяти, способны (и должны) использовать классы `vector` или `string`, если им нужно управляемое хранение. Классы, использующие векторы и строки, избегают сложностей, связанных с резервированием и освобождением памяти.

Кроме того, синтезируемые версии операторов копирования, присвоения и удаления правильно работают для классов, у которых есть переменные-члены класса `vector` или `string`. При копировании или присвоении объекта, обладающего переменной-членом класса `vector`, этот класс сам позаботится

о копировании и присвоении своих элементов. Когда объект удаляется, переменная-член класса `vector` тоже удаляется, что в свою очередь удаляет элементы вектора. Класс `string` работает аналогично.



Пока вы еще не знаете, как определить операторы, описанные в главе 13, ресурсы, резервируемые вашими классами, должны храниться непосредственно как переменные-члены класса.



7.2. Управление доступом и инкапсуляция

На настоящий момент для нашего класса определен интерфейс; однако ничто не вынуждает пользователей использовать его. Наш класс еще не использует инкапсуляцию — пользователи вполне могут обратиться к объекту `Sales_data` и воспользоваться его реализацией. Для обеспечения инкапсуляции в языке C++ используют *спецификаторы доступа* (access specifier).

- Члены класса, определенные после спецификатора `public`, доступны для всех частей программы. *Открытые члены* (public member) определяют интерфейс к классу.
- Члены, определенные после спецификатора `private`, являются *закрытыми* (private member), они доступны для функций-членов класса, но не доступны для кода, который использует класс. Разделы `private` инкапсулируют (т.е. скрывают) реализацию.

Переопределив класс `Sales_data` еще раз, получаем следующее:

```
class Sales_data {
public:           // добавлен спецификатор доступа
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(std::istream&);
    std::string isbn() const { return bookNo; }
    Sales_data &combine(const Sales_data&);
private:       // добавлен спецификатор доступа
    double avg_price() const
    { return units_sold ? revenue/units_sold : 0; }
    std::string bookNo;
    unsigned units_sold = 0;
    double revenue = 0.0;
};
```

Конструкторы и функции-члены, являющиеся частью интерфейса (например, `isbn()` и `combine()`), должны располагаться за спецификатором `public`; переменные-члены и функции, являющиеся частью реализации, располагаются за спецификатором `private`.

Класс может содержать любое количество спецификаторов доступа; нет никаких ограничений на то, как часто используется спецификатор. Каждый спецификатор определяет уровень доступа последующих членов. Заданный уровень доступа остается в силе до следующего спецификатора доступа или до конца тела класса.

Использование ключевых слов `class` и `struct`

Было также внесено еще одно изменение: в начале определения класса использовано *ключевое слово* `class`, а не `struct`. Это изменение является чисто стилистическим; тип класса можно определить при помощи любого из этих ключевых слов. Единственное различие между ключевыми словами `struct` и `class` в заданном по умолчанию уровне доступа.

Члены класса могут быть определены перед первым спецификатором доступа. Уровень доступа к таким членам будет зависеть от того, как определяется класс. Если используется ключевое слово `struct`, то члены, определенные до первого спецификатора доступа, будут открытыми; если используется ключевое слово `class`, то они будут закрытыми.

Общепринятым стилем считается определение классов, все члены которого предположительно будут открытыми, с использованием ключевого слова `struct`. Если члены класса должны быть закрытыми, используется ключевое слово `class`.



Единственное различие между ключевыми словами `class` и `struct` в задаваемом по умолчанию уровне доступа.

Ключевая концепция. Преимущества инкапсуляции

Инкапсуляция предоставляет два важных преимущества.

- Пользовательский код не может по неосторожности повредить состояние инкапсулированного объекта.
- Реализация инкапсулированного класса может со временем измениться, это не потребует изменений в коде на пользовательском уровне.

Определив переменные-члены закрытыми, автор класса получает возможность вносить изменения в данные. Если реализация изменится, то вызванные этим последствия можно исследовать только в коде класса. Пользовательский код придется изменять только при изменении интерфейса. Если данные являются открытыми, то любой использовавший их код может быть нарушен. Пришлось бы найти и переписать любой код, который полагался на прежнюю реализацию, и только затем использовать программу.

Еще одно преимущество объявления переменных-членов закрытыми в том, что данные защищены от ошибок, которые могли бы внести пользователи. Если есть ошибка, повреждающая состояние объекта, места ее поиска ограничены только тем кодом, который является частью реализации. Это существенно облегчает поиск проблем и обслуживание программы.

Упражнения раздела 7.2

Упражнение 7.16. Каковы ограничения (если они есть) на количество спецификаторов доступа в определении класса? Какие виды членов должны быть определены после спецификатора `public`? Какие после спецификатора `private`?

Упражнение 7.17. Каковы различия (если они есть) между ключевыми словами `class` и `struct`?

Упражнение 7.18. Что такое инкапсуляция? Чем она полезна?

Упражнение 7.19. Укажите, какие члены класса `Person` имеет смысл объявить как `public`, а какие как `private`. Объясните свой выбор.



7.2.1. Друзья

Теперь, когда переменные-члены класса `Sales_data` стали закрытыми, функции `read()`, `print()` и `add()` перестали компилироваться. Проблема в том, что хоть эти функции и являются частью интерфейса класса `Sales_data`, его членами они не являются.

Класс может позволить другому классу или функции получить доступ к своим не открытым членам, установив для них *дружественные отношения* (`friend`). Класс объявляет функцию дружественной, включив ее объявление с предваряющим ключевым словом `friend`:

```
class Sales_data {
// добавлены объявления дружественных функций, не являющихся
// членами класса Sales_data
friend Sales_data add(const Sales_data&, const Sales_data&);
friend std::istream &read(std::istream&, Sales_data&);
friend std::ostream &print(std::ostream&, const Sales_data&);
// другие члены и спецификаторы доступа, как прежде
public:
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    Sales_data(const std::string &s): bookNo(s) { }
    Sales_data(std::istream&);
    std::string isbn() const { return bookNo; }
    Sales_data &combine(const Sales_data&);
private:
```

```

std::string bookNo;
unsigned units_sold = 0;
double revenue = 0.0;
};
// объявления частей, не являющихся членами интерфейса класса Sales_data
Sales_data add(const Sales_data&, const Sales_data&);
std::istream &read(std::istream&, Sales_data&);
std::ostream &print(std::ostream&, const Sales_data&);

```

Объявления друзей могут располагаться только в определении класса; использовать они могут в классе повсюду. Друзья не являются членами класса и не подчиняются спецификаторам доступа раздела, в котором они объявлены. Более подробная информация о дружественных отношениях приведена в разделе 7.3.4 (стр. 363).



Объявления друзей имеет смысл группировать в начале или в конце определения класса.



Хотя пользовательский код не должен изменяться при изменении определения класса, файлы исходного кода, использующие этот класс, следует перекомпилировать при каждом изменении класса.



Объявление дружественных отношений

Объявление дружественных отношений устанавливает только право доступа. Это не объявление функции. Если необходимо, чтобы пользователи класса были в состоянии вызвать дружественную функцию, ее следует также объявить.

Чтобы сделать друзей класса видимыми его пользователям, их обычно объявляют вне класса в том же заголовке, что и сам класс. Таким образом, в заголовке `Sales_data` следует предоставить отдельные объявления (кроме объявлений дружественными в теле класса) для функций `read()`, `print()` и `add()`.



Многие компиляторы не выполняют правило, согласно которому дружественные функции должны быть объявлены *вне* класса, прежде чем они будут применены.

Некоторые компиляторы позволяют вызвать дружественную функцию, когда для нее нет обычного объявления. Даже если ваш компилятор позволяет такие вызовы, имеет смысл предоставлять отдельные объявления для дружественных функций. Так не придется переделывать весь код, если вы перейдете на компилятор, который выполняет это правило.

Упражнения раздела 7.2.1

Упражнение 7.20. Когда полезны дружественные отношения? Укажите преимущества и недостатки их использования.

Упражнение 7.21. Измените свой класс `Sales_data` так, чтобы скрыть его реализацию. Написанные вами программы, которые использовали операции класса `Sales_data`, должны продолжить работать. Перекомпилируйте эти программы с новым определением класса, чтобы проверить, остались ли они работоспособными.

Упражнение 7.22. Измените свой класс `Person` так, чтобы скрыть его реализацию.

7.3. Дополнительные средства класса

Хотя класс `Sales_data` довольно прост, он все же позволил исследовать немало средств поддержки классов. В этом разделе рассматриваются некоторые из дополнительных средств, связанных с классом, которые класс `Sales_data` не будет использовать. К этим средствам относятся *типы-члены* (type member), внутри-классовые инициализаторы для типов-членов класса, изменяемые переменные-члены, встраиваемые функции-члены, функции-члены, возвращающие `*this`, а также подробности определения и использования типов класса и дружественных классов.

7.3.1. Снова о членах класса

Для исследования некоторых из дополнительных средств определим пару взаимодействующих классов по имени `Screen` и `Window_mgr`.

Определение типов-членов

Класс `Screen` представляет окно на экране. У каждого объекта класса `Screen` есть переменная-член типа `string`, хранящая содержимое окна и три переменные-члена типа `string::size_type`, представляющие позицию курсора, высоту и ширину окна.

Кроме переменных и функций-членов, класс может определять собственные локальные имена таких типов. Определенные классом имена типов подчиняются тем же правилам доступа, что и любой другой его член, и могут быть открытыми или закрытыми:

```
class Screen {
public:
    typedef std::string::size_type pos;
private:
    pos cursor = 0;
    pos height = 0, width = 0;
    std::string contents;
};
```

Тип `pos` определен в части `public` класса `Screen`, поскольку пользователи должны использовать это имя. Пользователи класса `Screen` не обязаны знать, что он использует класс `string` для хранения своих данных. Определив тип `pos` как открытый член, эту подробность реализации класса `Screen` можно скрыть.

В объявлении типа `pos` есть два интересных момента. Во-первых, хоть здесь и был использован оператор `typedef` (см. раздел 2.5.1, стр. 106), с таким же успехом можно использовать псевдоним типа (см. раздел 2.5.1, стр. 106):

```
class Screen {
public:
    // альтернативный способ объявления типа-члена с использованием
    // псевдонима типа
    using pos = std::string::size_type;
    // другие члены как прежде
};
```

Во-вторых, по причинам, которые будут описаны в разделе 7.3.4 (стр. 368), в отличие от обычных членов, типы-члены определяются прежде, чем используются. В результате типы-члены обычно располагают в начале класса.

Функции-члены класса `Screen`

Чтобы сделать наш класс полезней, добавим в него конструктор, позволяющий пользователям задавать размер и содержимое экрана, наряду с членами, позволяющими переместить курсор и получить символ в указанной позиции:

```
class Screen {
public:
    typedef std::string::size_type pos;
    Screen() = default; // необходим, поскольку у класса Screen есть
                        // другой конструктор
    // внутриклассовый инициализатор инициализирует курсор значением 0
    Screen(pos ht, pos wd, char c): height(ht), width(wd),
                                   contents(ht * wd, c) { }

    char get() const // получить символ в курсоре
    { return contents[cursor]; } // неявно встраиваемая
    inline char get(pos ht, pos wd) const; // явно встраиваемая
    Screen &move(pos r, pos c); // может быть сделана встраиваемой позже
private:
    pos cursor = 0;
    pos height = 0, width = 0;
    std::string contents;
};
```

Поскольку мы предоставляем конструктор, компилятор не будет автоматически создавать стандартный конструктор сам. Если у нашего класса должен быть стандартный конструктор, то придется создать его явно. В данном случае используется синтаксис `= default`, чтобы попросить компилятор самому создать определение стандартного конструктора (см. раздел 7.1.4, стр. 345).

Стоит также обратить внимание на то, что второй конструктор (получающий три аргумента) неявно использует внутриклассовый инициализатор для переменной-члена `cursor` (см. раздел 7.1.4, стр. 346). Если бы у класса не было внутриклассового инициализатора для переменной-члена `cursor`, то мы явно инициализировали бы ее наряду с другими переменными-членами.

Встраиваемые члены класса

У классов зачастую бывают небольшие функции, которые выгодно сделать встраиваемыми. Как уже упоминалось, определенные в классе функции-члены автоматически являются встраиваемыми (`inline`) (см. раздел 6.5.2, стр. 312). Таким образом, конструкторы класса `Screen` и версия функции `get()`, возвращающей обозначенный курсором символ, являются встраиваемыми по умолчанию.

Функцию-член можно объявить встраиваемой явно в ее объявлении в теле класса. В качестве альтернативы функцию можно указать встраиваемой в определении, расположенном вне тела класса:

```
inline // функцию можно указать встраиваемой в определении
Screen &Screen::move(pos r, pos c)
{
    pos row = r * width; // вычислить положение ряда
    cursor = row + c;    // переместить курсор к столбцу этого ряда
    return *this;        // вернуть этот объект как l-значение
}

char Screen::get(pos r, pos c) const // объявить встраиваемый в классе
{
    pos row = r * width; // вычислить положение ряда
    return contents[row + c]; // вернуть символ в данном столбце
}
```

Хоть и не обязательно делать это, вполне допустимо указать ключевое слово `inline` и в объявлении, и в определении. Однако указание ключевого слова `inline` в определении только вне класса может облегчить чтение класса.



По тем же причинам, по которым встраиваемые функции определяются в заголовках (см. раздел 1.1, стр. 313), встраиваемые функции-члены следует определить в том же заголовке, что и определение соответствующего класса.

Перегрузка функций-членов

Подобно функциям, которые не являются членами класса, функции-члены могут быть перегружены (см. раздел 6.4, стр. 302), если они отличаются количеством и/или типами параметров. При вызове функции-члена используется тот же процесс подбора функции (см. раздел 6.4, стр. 305), что и у функций, не являющихся членом класса.

Например, в классе `Screen` определены две версии функции `get()`. Одна версия возвращает символ, обозначенный в настоящее время курсором; другая возвращает символ в указанной позиции, определенной ее рядом и столбцом. Чтобы определить применяемую версию, компилятор использует количество аргументов:

```
Screen myscreen;
char ch = myscreen.get(); // вызов Screen::get()
ch = myscreen.get(0,0);  // вызов Screen::get(pos, pos)
```

Изменяемые переменные-члены

Иногда (но не очень часто) у класса есть переменная-член, которую следует сделать изменяемой даже в константной функции-члене. Для обозначения таких членов в их объявление включают ключевое слово `mutable`.

Изменяемая переменная-член (`mutable data member`) никогда не бывает константой, даже когда это член константного объекта. Соответственно константная функция-член может изменить изменяемую переменную-член. В качестве примера добавим в класс `Screen` изменяемую переменную-член `access_ctr`, используемую для отслеживания частоты вызова каждой функции-члена класса `Screen`:

```
class Screen {
public:
    void some_member() const;
private:
    mutable size_t access_ctr; // может измениться даже в константном
                               // объекте
    // другие члены как прежде
};
void Screen::some_member() const
{
    ++access_ctr; // сохранить количество вызовов любой функции-члена
                 // безотносительно других выполняемых ею действий
}
```

Несмотря на то что функция-член `some_member()` константная, она может изменить значение переменной-члена `access_ctr`. Этот член класса является изменяемым, поэтому любая функция-член, включая константные, может изменить это значение.

Инициализаторы переменных-членов класса

C++ 11 Кроме класса `Screen`, определим также класс диспетчера окон, который представляет коллекцию окон на данном экране. У этого класса будет вектор объектов класса `Screen`, каждый элемент которого представляет отдельное окно. По умолчанию класс `Window_mgr` должен изначально содержать один объект класса `Screen`, инициализированный значением по умолчанию. По новому стандарту наилучшим способом определения

такого значения по умолчанию является внутриклассовый инициализатор (см. раздел 2.6.1, стр. 113):

```
class Window_mgr {  
private:  
    // по умолчанию отслеживающий окна объект класса Window_mgr  
    // содержит одно пустое окно стандартного размера  
    std::vector<Screen> screens{Screen(24, 80, ' ')};  
};
```

При инициализации переменных-членов типа класса их конструктору следует предоставить аргументы. В этом случае применяется список инициализации переменной-члена типа `vector` (см. раздел 3.3.1, стр. 143) с инициализатором для одного элемента. Этот инициализатор содержит значение типа `Screen`, передаваемое конструктору `vector<Screen>` для создания вектора с одним элементом. Это значение создается конструктором класса `Screen`, получающим параметры в виде двух размерностей и заполняющего символа, чтобы создать пустое окно заданного размера.

Как уже упоминалось, для внутриклассовой инициализации может использоваться форма инициализации `=` (как при инициализации переменных-членов класса `Screen`) или прямая форма инициализации с использованием фигурных скобок (как у вектора `screens`).



При предоставлении внутриклассового инициализатора это следует сделать после знака `=` или в фигурных скобках.

Упражнения раздела 7.3.1

Упражнение 7.23. Напишите собственную версию класса `Screen`.

Упражнение 7.24. Добавьте в свой класс `Screen` три конструктора: стандартный; получающий высоту, ширину и заполняющий содержимое соответствующим количеством пробелов; получающий высоту, ширину и заполняющий символ для содержимого экрана.

Упражнение 7.25. Может ли класс `Screen` безопасно полагаться на заданные по умолчанию версии операторов копирования и присвоения? Если да, то почему? Если нет, то почему?

Упражнение 7.26. Определите функцию `Sales_data::avg_price` как встраиваемую.



7.3.2. Функции, возвращающие указатель `*this`

Теперь добавим функции, устанавливающие символ в курсоре или в заданной области:

```

class Screen {
public:
    Screen &set(char);
    Screen &set(pos, pos, char);
    // другие члены, как прежде
};
inline Screen &Screen::set(char c)
{
    contents[cursor] = c; // установите новое значение в текущей позиции
                          // курсора
    return *this;        // вернуть этот объект как l-значение
}
inline Screen &Screen::set(pos r, pos col, char ch)
{
    contents[r*width + col] = ch; // установить позицию по данному
                                  // значению
    return *this;                // вернуть этот объект как l-значение
}

```

Как и функция `move()`, функция-член `set()` возвращает ссылку на объект, из которого они вызваны (см. раздел 7.1.2, стр. 339). Возвращающие ссылку функции являются l-значениями (см. раздел 6.3.2, стр. 296), а это означает, что они возвращают сам объект, а не его копию. Это позволяет связать несколько их вызовов в одно выражение:

```

// переместить курсор в указанную позицию и присвоить
// символу значение
myScreen.move(4,0).set('#');

```

Эти операции выполняются для того же объекта. В этом выражении сначала перемещается курсор (`move()`) в окно (`myScreen`), а затем устанавливается (`set()`) заданный символ. Таким образом, этот оператор эквивалентен следующему:

```

myScreen.move(4,0);
myScreen.set('#');

```

Если бы функции `move()` и `set()` возвращали тип `Screen`, а не `Screen&`, этот оператор выполнялся бы совсем по-другому. В данном случае он был бы эквивалентен следующему:

```

// если move возвращает Screen, а не Screen&
Screen temp = myScreen.move(4,0); // возвращаемое значение было
                                  // бы скопировано
temp.set('#'); // содержимое myScreen осталось бы неизменно

```

Если бы функция `move()` имела возвращаемое значение не ссылочного типа, то оно было бы копией `*this` (см. раздел 6.3.2, стр. 294). Вызов функции `set()` изменил бы лишь временную копию, а не сам объект `myScreen`.

Возвращение `*this` из константной функции-члена

Теперь добавим функцию `display()`, выводящую содержимое окна. Необходима возможность включать эту операцию в последовательность операций `set()` и `move()`. Поэтому, подобно функциям `set()` и `move()`, функция `display()` возвратит ссылку на объект, для которого она выполняется.

Логически отображение объекта класса `Screen` (окна) не изменяет его, поэтому функцию `display()` следует сделать константным членом. Но если функция `display()` будет константной, то `this` будет указателем на константу, а значение `*this` — константным объектом. Следовательно, типом возвращаемого значения функции `display()` будет `const Sales_data&`. Однако, если функция `display()` возвратит ссылку на константу, мы не сможем вставить вызов функции `display()` в последовательность действий:

```
Screen myScreen;
// если display возвращает константную ссылку, вызов в последовательности
// будет ошибкой
myScreen.display(cout).set('*');
```

Хотя объект `myScreen` неконстантный, вызов функции `set()` не будет компилироваться. Проблема в том, что константная версия функции `display()` возвращает ссылку на константу, и мы не можем вызвать функцию `set()` для константного объекта.



Тип возвращаемого значения константной функции-члена, возвращающей `*this` как ссылку, должен быть ссылкой на константу.

Перегрузка на основании константности

Функции-члены вполне можно перегружать исходя из того, являются ли они константными или нет, причем по тем же причинам, по которым функцию можно перегружать исходя из того, является ли ее параметр указателем на константу (см. раздел 6.4, стр. 305). Неконстантная версия неприменима для константных объектов; она применима только для константных объектов. Для неконстантного объекта можно вызвать любую версию, но неконстантная версия будет лучшим соответствием.

В этом примере определим закрытую функцию-член `do_display()` для фактического вывода окна. Каждая из функций `display()` вызовет эту функцию, а затем возвратит объект, для которого она выполняется:

```
class Screen {
public:
    // display перегружена на основании того, является ли
    // объект константой или нет
    Screen &display(std::ostream &os)
        { do_display(os); return *this; }
```

```

    const Screen &display(std::ostream &os) const
        { do_display(os); return *this; }
private:
    // функция отображения окна
    void do_display(std::ostream &os) const {os << contents;}
    // другие члены как прежде
};

```

Как и в любом другом случае, при вызове одной функции-члена другой неявно передается указатель `this`. Таким образом, когда функция `display()` вызывает функцию-член `do_display()`, ей неявно передается собственный указатель `this`. Когда неконстантная версия функции `display()` вызывает функцию `do_display()`, ее указатель `this` неявно преобразуется из указателя на неконстанту в указатель на константу (см. раздел 4.11.2, стр. 222).

Когда функция `do_display()` завершает работу, функция `display()` возвращает объект, с которым они работают, обращаясь к значению указателя `this`. В неконстантной версии указатель `this` указывает на неконстантный объект, так что эта версия функции `display()` возвращает обычную, неконстантную ссылку; константная версия возвращает ссылку на константу.

Когда происходит вызов функции `display()` для объекта, вызываемую версию определяет его константность:

```

Screen myScreen(5,3);
const Screen blank(5, 3);
myScreen.set('#').display(cout); // вызов неконстантной версии
blank.display(cout);           // вызов константной версии

```

СОВЕТ. ИСПОЛЬЗУЙТЕ ЗАКРЫТЫЕ ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ

Некоторые читатели могут удивиться: зачем дополнительно создавать отдельную функцию `do_display()`? В конце концов, обращение к функции `do_display()` не намного проще, чем осуществляемое в ней действие. Зачем же она нужна? Причин здесь несколько.

- Всегда желательно избегать нескольких экземпляров одного кода.
- По мере развития класса функция `display()` может стать значительно более сложной, а следовательно, преимущества одой, а не нескольких копий кода станут более очевидными.
- Во время разработки в тело функции `display()`, вероятно, придется добавить отладочный код, который в финальной версии будет удален. Это будет проще сделать в случае, когда весь отладочный код находится в одной функции `do_display()`
- Поскольку функция `do_display()` объявлена встраиваемой (`inline`), при создании исполняемого кода компилятор и так вставит ее содержи-

мое по месту вызова, поэтому вызов функции не повлечет за собой никаких потерь времени и ресурсов.

Обычно в хорошо спроектированных программах на языке C++ присутствует множество маленьких функций, таких как `do_display()`, которые выполняют всю основную работу, когда их использует набор других функций.

Упражнения раздела 7.3.2

Упражнение 7.27. Добавьте функции `move()`, `set()` и `display()` в свою версию класса `Screen`. Проверьте свой класс, выполнив следующий код:

```
Screen myScreen(5, 5, 'X');
myScreen.move(4,0).set('#').display(cout);
cout << "\n";
myScreen.display(cout);
cout << "\n";
```

Упражнение 7.28. Что если бы в предыдущем упражнении типом возвращаемого значения функций `move()`, `set()` и `display()` был `Screen`, а не `Screen&`?

Упражнение 7.29. Пересмотрите свой класс `Screen` так, чтобы функции `move()`, `set()` и `display()` возвращали тип `Screen`, а затем проверьте свое предположение из предыдущего упражнения.

Упражнение 7.30. Обращение к членам класса при помощи указателя `this` вполне допустимо, но избыточно. Обсудите преимущества и недостатки явного использования указателя `this` для доступа к членам.

7.3.3. Типы классов

Каждый класс определяет уникальный тип. Два различных класса определяют два разных типа, даже если их члены совпадают. Например:

```
struct First {
    int mem1;
    int getMem();
};
struct Second {
    int mem1;
    int getMem();
};
First obj1;
Second obj2 = obj1; // ошибка: obj1 и obj2 имеют разные типы
```



Даже если у двух классов полностью совпадает список членов, они являются разными типами. Члены каждого класса отличны от членов любого другого класса (или любой другой области видимости).

К типу класса можно обратиться непосредственно, используя имя класса как имя типа. В качестве альтернативы можно использовать имя класса после ключевого слова `class` или `struct`:

```
Sales_data item1;           // инициализация значением по умолчанию объекта
                             // типа Sales_data
class Sales_data item1;     // эквивалентное объявление
```

Оба способа обращения к типу класса эквивалентны. Второй метод унаследован от языка C и все еще допустим в C++.

Объявления класса

Подобно тому, как можно объявить функцию без ее определения (см. раздел 6.1.2, стр. 273), можно *объявить* (class declaration) класс, не определяя его:

```
class Screen; // объявление класса Screen
```

Такое объявление иногда называют *предварительным объявлением* (forward declaration), оно вводит имя `Screen` в программу и указывает, что оно относится к типу класса. После объявления, но до определения, тип `Screen` считается *незавершенным типом* (incomplete type), т.е. известно, что `Screen` — это тип класса, но не известно, какие члены он содержит.

Использование незавершенного типа весьма ограничено. Его можно использовать только для определения указателей или ссылок, а также для объявления (но не определения) функций, которые используют этот тип как параметр или тип возвращаемого значения.

Прежде чем можно будет писать код, создающий объекты некоего класса, его следует определить, а не только объявить. В противном случае компилятор не будет знать, в каком объеме памяти нуждаются его объекты. Аналогично класс должен быть уже определен перед использованием ссылки или указателя для доступа к члену класса. В конце концов, если класс не был определен, компилятор не сможет узнать, какие члены имеет класс.

За одним исключением, рассматриваемым в разделе 7.6 (стр. 387), переменные-члены могут быть определены как имеющие тип класса, только если класс был определен. Тип следует завершить, поскольку компилятор должен знать объем памяти, необходимый для хранения переменных-членов. Пока класс не определен, пока его тело не создано, у класса не может быть переменных-членов его собственного типа. Однако класс считается объявленным (но еще не определенным), как только его имя стало видимо. Поэтому у класса могут быть переменные-члены, являющиеся указателями или ссылками на ее собственный тип:

```
class Link_screen {
    Screen window;
    Link_screen *next;
    Link_screen *prev;
};
```

Упражнения раздела 7.3.3

Упражнение 7.31. Определите два класса, X и Y , у которых класс X имеет указатель на класс Y , а Y содержит объект типа X .

7.3.4. Снова о дружественных отношениях

Наш класс `Sales_data` определил три обычных функции, не являющиеся членом класса, как дружественные (см. раздел 7.2.1, стр. 351). Класс может также сделать дружественным другой класс или объявить дружественными определенные функции-члены другого (определенного ранее) класса. Кроме того, дружественная функция может быть определена в теле класса. Такие функции неявно являются встраиваемыми.

Дружественные отношения между классами

В качестве примера дружественных классов рассмотрим класс `Window_mgr` (см. раздел 7.3.1, стр. 357), его членам понадобится доступ к внутренним данным объектов класса `Screen`, которыми они управляют. Предположим, например, что в класс `Window_mgr` необходимо добавить функцию-член `clear()`, заполняющую содержимое определенного окна пробелами. Для этого функции `clear()` нужен доступ к закрытым переменным-членам класса `Screen`. Для этого класс `Screen` должен объявить класс `Window_mgr` дружественным:

```
class Screen {
    // члены класса Window_Mgr смогут обращаться к закрытым
    // членам класса Screen
    friend class Window_mgr;
    // ... остальное, как раньше в классе Screen
};
```

Функции-члены дружественного класса могут обращаться ко всем членам класса, объявившего его другом, включая не открытые члены. Теперь, когда класс `Window_mgr` является другом класса `Screen`, функцию-член `clear()` класса `Window_mgr` можно переписать следующим образом:

```
class Window_mgr {
public:
    // идентификатор области для каждого окна на экране
    using ScreenIndex = std::vector<Screen>::size_type;
    // сбросить данное окно, заполнив его пробелами
    void clear(ScreenIndex);
private:
    std::vector<Screen> screens{Screen(24, 80, ' ')};
};
void Window_mgr::clear(ScreenIndex i)
{
    // s - ссылка на окно, которое предстоит очистить
```

```

Screen &s = screens[i];
// сбросить данное окно, заполнив его пробелами
s.contents = string(s.height * s.width, ' ');
}

```

Сначала определим `s` как ссылку на класс `Screen` в позиции `i` вектора окон. Затем переменные-члены `height` и `width` данного объекта класса `Screen` используются для вычисления количества символов новой строки, содержащей пробелы. Эта заполненная пробелами строка присваивается переменной-члену `contents`.

Если бы функция `clear()` не была дружественной классу `Screen`, то этот код не компилировался бы. Функция `clear()` не смогла бы использовать переменные-члены `height`, `width` или `contents` класса `Screen`. Поскольку класс `Screen` установил дружественные отношения с классом `Window_mgr`, для его функций доступны все члены класса `Screen`.

Важно понимать, что дружественные отношения не передаются. Таким образом, если у класса `Window_mgr` есть собственные друзья, то у них нет привилегий доступа к членам класса `Screen`.



Каждый класс сам контролирует, какие классы или функции будут его друзьями.

Как сделать функцию-член дружественной

Вместо того чтобы делать весь класс `Window_mgr` дружественным классу `Screen`, можно предоставить доступ только функции-члену `clear()`. Когда функция-член объявляется дружественной, следует указать класс, которому она принадлежит:

```

class Screen {
    // класс Window_mgr::clear должен быть объявлен перед классом Screen
    friend void Window_mgr::clear(ScreenIndex);
    // ... остальное как раньше в классе Screen
};

```

Создание дружественных функций-членов требует тщательного структурирования программ в соответствии с взаимозависимостями объявлений и определений. В данном случае программу следует упорядочить следующим образом.

- Сначала определите класс `Window_mgr`, который объявляет, но не может определить функцию `clear()`. Класс `Screen` должен быть объявлен до того, как функция `clear()` сможет использовать члены класса `Screen`.
- Затем определите класс `Screen`, включая объявление функции `clear()` дружественной.
- И наконец, определите функцию `clear()`, способную теперь обращаться к членам класса `Screen`.

Перегруженные функции и дружественные отношения

Хотя у перегруженных функций одинаковое имя, это все же разные функции. Поэтому класс должен объявить дружественной каждую из перегруженных функций:

```
// перегруженные функции storeOn
extern std::ostream& storeOn(std::ostream &, Screen &);
extern BitMap& storeOn(BitMap &, Screen &);
class Screen {
    // версия ostream функции storeOn может обращаться к закрытым членам
    // объектов класса Screen
    friend std::ostream& storeOn(std::ostream &, Screen &);
    // ...
};
```

Класс Screen объявляет другом версию функции storeOn, получающей поток ostream&. Версия, получающая параметр BitMap&, особых прав доступа к объектам класса Screen не имеет.



Объявление дружественных отношений и область видимости

Классы и функции, не являющиеся членами класса, не следует объявлять прежде, чем они будут использованы в объявлении дружественными. Когда имя впервые появляется в объявлении дружественной, оно неявно *подразумевается* принадлежащей окружающей области видимости. Однако сам друг фактически не объявлен в этой области видимости (см. раздел 7.2.1, стр. 352).

Даже если мы определим функцию в классе, ее все равно придется объявить за пределами класса, чтобы сделать видимой. Объявление должно уже существовать, даже если вызывается дружественная функция:

```
struct X {
    friend void f() { /* дружественная функция может быть определена в теле
    класса */ }
    X() { f(); } // ошибка: нет объявления для f
    void g();
    void h();
};
void X::g() { return f(); } // ошибка: f не была объявлена
void f(); // объявляет функцию, определенную в X
void X::h() { return f(); } // ок: объявление f теперь в области
// видимости
```

Важно понимать, что объявление дружественной затрагивает доступ, но не является объявлением в обычном смысле.



Помните: некоторые компиляторы не выполняют правил поиска имен друзей (см. раздел 7.2.1, стр. 352).

Упражнения раздела 7.3.4

Упражнение 7.32. Определите собственные версии классов `Screen` и `Window_mgr`, в которых функция `clear()` является членом класса `Window_mgr` и другом класса `Screen`.



7.4. Область видимости класса

Каждый класс определяет собственную область видимости. Вне области видимости класса (class scope) к обычным данным и функциям его члены могут обращаться только через объект, ссылку или указатель, используя оператор доступа к члену (см. раздел 4.6, стр. 207). Для доступа к членам типа из класса используется оператор области видимости. В любом случае следующее за оператором имя должно быть членом соответствующего класса.

```
Screen::pos ht = 24, wd = 80; // использование типа pos, определенного
                               // в классе Screen

Screen scr(ht, wd, ' ');
Screen *p = &scr;
char c = scr.get(); // доступ к члену get() объекта scr
c = p->get();        // доступ к члену get() из объекта, на который
                     // указывает p
```

Область видимости и члены, определенные вне класса

Тот факт, что класс определяет область видимости, объясняет, почему следует предоставить имя класса наравне с именем функции, при определении функции-члена вне ее класса (см. раздел 7.1.2, стр. 339). За пределами класса имена ее членов скрыты.

Как только имя класса становится видимо, остальная часть определения, включая список параметров и тело функции, находится в области видимости класса. В результате мы можем обращаться к другим членам класса без уточнения.

Вернемся, например, к функции-члену `clear()` класса `Window_mgr` (см. раздел 7.3.4, стр. 363). Параметр этой функции имеет тип, определенный в классе `Window_mgr`:

```
void Window_mgr::clear(ScreenIndex i)
{
    Screen &s = screens[i];
    s.contents = string(s.height * s.width, ' ');
}
```

Поскольку компилятор видит последующий список параметров и ничего подобного в области видимости класса `WindowMgr`, нет никакой необходимости определять, что требуется тип `ScreenIndex`, определенный в классе

WindowMgr. По той же причине использование объекта `screens` в теле функции относится к имени, объявленному в классе `Window_mgr`.

С другой стороны, тип возвращаемого значения функции обычно располагается перед именем функции. Когда функция-член определяется вне тела класса, любое имя, используемое в типе возвращаемого значения, находится вне области видимости класса. В результате тип возвращаемого значения должен определять класс, членом которого он является. Например, мы могли бы добавить в класс `Window_mgr` функцию `addScreen()`, добавляющую еще одно окно на экран. Этот член класса возвратит значение типа `ScreenIndex`, которое пользователь впоследствии сможет использовать для поиска этого окна:

```
class Window_mgr {
public:
    // добавить окно на экран и вернуть его индекс
    ScreenIndex addScreen(const Screen&);
    // другие члены, как прежде
};
// тип возвращаемого значения видим прежде, чем начинается область
// видимости класса Window_mgr
Window_mgr::ScreenIndex
Window_mgr::addScreen(const Screen &s)
{
    screens.push_back(s);
    return screens.size() - 1;
}
```

Поскольку тип возвращаемого значения встречается прежде имени класса, оно находится вне области видимости класса `Window_mgr`. Чтобы использовать тип `ScreenIndex` для возвращаемого значения, следует определить класс, в котором определяется этот тип.

Упражнения раздела 7.4

Упражнение 7.33. Что будет, если добавить в класс `Screen` переменную-член `size()`, определенную следующим образом? Исправьте все обнаруженные ошибки.

```
pos Screen::size() const
{
    return height * width;
}
```



7.4.1. Поиск имен в области видимости класса

В рассмотренных до сих пор программах *поиск имен* (name lookup) (процесс поиска объявления, соответствующего данному имени) был относительно прост.

- Сначала поиск объявления осуществляется в том блоке кода, в котором используется имя. Причем рассматриваются только те имена, объявления которых расположены перед местом применения.
- Если имя не найдено, поиск продолжается в иерархии областей видимости, начиная с текущей.
- Если объявление так и не найдено, происходит ошибка.

Когда поиск имен осуществляется в функциях-членах, определенных в классе, может показаться, что он происходит не по правилам поиска. Но в данном случае внешний вид обманчив. Обработка определений классов осуществляется в два этапа.

- Сначала компилируются объявления членов класса.
- Тела функции компилируются только после того, как виден весь класс.



Определения функций-членов обрабатываются *после того*, как компилятор обработает все объявления в классе.

Классы обрабатываются в два этапа, чтобы облегчить организацию кода класса. Поскольку тела функций-членов не обрабатываются, пока весь класс не станет видимым, они смогут использовать любое имя, определенное в классе. Если бы определения функций обрабатывались одновременно с объявлениями переменных-членов, то пришлось бы располагать функции-члены так, чтобы они обращались только к тем именам, которые уже видимы.

Поиск имен для объявлений членов класса

Этот двухэтапный процесс применяется только к именам, используемым в теле функции-члена. Имена, используемые в объявлениях, включая имя типа возвращаемого значения и типов списка параметров, должны стать видимы прежде, чем они будут использованы. Если объявление переменной-члена будет использовать имя, объявление которого еще не видимо в классе, то компилятор будет искать то имя в той области (областях) видимости, в которой определяется класс. Рассмотрим пример.

```
typedef double Money;
string bal;
class Account {
public:
    Money balance() { return bal; }
private:
    Money bal;
    // ...
};
```

Когда компилятор видит объявление функции `balance()`, он ищет объявление имени `Money` в классе `Account`. Компилятор рассматривает только те

объявления в классе `Account`, которые расположены перед использованием имени `Money`. Поскольку его объявление как члена класса не найдено, компилятор ищет имя в окружающей области видимости. В этом примере компилятор найдет определение типа (`typedef`) `Money`. Этот тип будет использоваться и для типа возвращаемого значения функции `balance()`, и как тип переменной-члена `bal`. С другой стороны, тело функции `balance()` обрабатывается только после того, как видимым становится весь класс. Таким образом, оператор `return` в этой функции возвращает переменную-член по имени `bal`, а не строку из внешней области видимости.

Имена типов имеют особенности

Обычно внутренняя область видимости может переопределить имя из внешней области видимости, даже если это имя уже использовалось во внутренней области видимости. Но если член класса использует имя из внешней области видимости и это имя типа, то класс не сможет впоследствии переопределить это имя:

```
typedef double Money;
class Account {
public:
    Money balance() { return bal; } // используется имя Money из внешней
                                   // область видимости
private:
    typedef double Money; // ошибка: нельзя переопределить Money
    Money bal;
    // ...
};
```

Следует заметить, что хотя определение типа `Money` в классе `Account` использует тот же тип, что и определение во внешней области видимости, этот код все же ошибочен.

Хотя переопределение имени типа является ошибкой, не все компиляторы обнаружат эту ошибку. Некоторые спокойно примут такой код, даже если программа ошибочна.



Определения имен типов обычно располагаются в начале класса. Так, любой член класса, который использует этот тип, будет расположен после определения его имени.

При поиске имен в областях видимости члены класса следуют обычным правилам

Поиск имени, используемого в теле функции-члена, осуществляется следующим образом.

- Сначала поиск объявления имени осуществляется в функции-члене. Как обычно, рассматриваются объявления в теле функции, только предшествующие месту использования имени.
- Если в функции-члене объявление не найдено, поиск продолжается в классе. Просматриваются все члены класса.
- Если объявление имени в классе не найдено, поиск продолжится в области видимости перед определением функции-члена.

Обычно не стоит использовать имя другого члена класса как имя параметра в функции-члене. Но для демонстрации поиска имени нарушим это правило в функции `dummy_fcn()`:

```
// обратите внимание: это сугубо демонстрационный код, отражающий плохую
// практику программирования. Обычно не стоит использовать одинаковое имя
// для параметра и функции-члена
int height; // определяет имя, впоследствии используемое в Screen
class Screen {
public:
    typedef std::string::size_type pos;
    void dummy_fcn(pos height) {
        cursor = width * height; // какое имя height имеется в виду?
    }
private:
    pos cursor = 0;
    pos height = 0, width = 0;
};
```

Когда компилятор обрабатывает выражение умножения в функции `dummy_fcn()`, он ищет имена сначала в пределах данной функции. Параметры функции находятся в области видимости функции. Таким образом, имя `height`, используемое в теле функции `dummy_fcn()`, принадлежит ее параметру.

В данном случае имя `height` параметра скрывает имя `height` переменной-члена класса. Если необходимо переопределить обычные правила поиска, то это можно сделать так:

```
// плохой подход: имена, локальные для функций-членов, не должны
// скрывать имена переменных-членов класса
void Screen::dummy_fcn(pos height) {
    cursor = width * this->height; // переменная-член height
    // альтернативный способ указания переменной-члена
    cursor = width * Screen::height; // переменная-член height
}
```



Несмотря на то что член класса скрыт, его все равно можно использовать. Достаточно указать его полное имя, включающее имя класса, либо явно применить указатель `this`.

Значительно проще обеспечить доступ к переменной-члену `height`, присвоив параметру другое имя:

```
// хороший подход: не используйте имена переменных-членов для
// параметров или других локальных переменных
void Screen::dummy_fcn(pos ht) {
    cursor = width * height; // переменная-член height
}
```

Теперь, когда компилятор будет искать имя `height`, в функции `dummy_fcn()` он его не найдет. Затем компилятор просмотрит класс `Screen`. Поскольку имя `height` используется в функции-члене `dummy_fcn()`, компилятор просмотрит все объявления членов класса. Несмотря на то что объявление имени `height` расположено после места его использования в функции `dummy_fcn()`, компилятор решает, что оно относится к переменной-члену `height`.

После поиска в области видимости класса продолжается поиск в окружающей области видимости

Если компилятор не находит имя в функции или в области видимости класса, он ищет его в окружающей области видимости. В данном случае имя `height` объявлено во внешней области видимости, перед определением класса `Screen`. Однако объект во внешней области видимости скрывается переменной-членом класса по имени `height`. Если необходимо имя из внешней области видимости, к нему можно обратиться явно, используя оператор области видимости:

```
// плохой подход: не скрывайте необходимые имена, которые
// определены в окружающих областях видимости
void Screen::dummy_fcn(pos height) {
    cursor = width * ::height; // который height? Глобальный
}
```



Несмотря на то что глобальный объект был скрыт, используя оператор области видимости, доступ к нему вполне можно получить.

Поиск имен распространяется по всему файлу, где они были применены

Когда член класса определен вне определения класса, третий этап поиска его имени происходит не только в объявлениях глобальной области видимости, которые расположены непосредственно перед определением класса `Screen`, но и распространяется на остальные объявления в глобальной области видимости. Рассмотрим пример.

```
int height; // определяет имя, впоследствии используемое в Screen
class Screen {
public:
    typedef std::string::size_type pos;
    void setHeight(pos);
    pos height = 0; // скрывает объявление height из внешней
                    // области видимости
};
```

```
Screen::pos verify(Screen::pos);
void Screen::setHeight(pos var) {
    // var: относится к параметру
    // height: относится к члену класса
    // verify: относится к глобальной функции
    height = verify(var);
}
```

Обратите внимание, что объявление глобальной функции `verify()` не видно перед определением класса `Screen`. Но третий этап поиска имени включает область видимости, в которой присутствует определение члена класса. В данном примере объявление функции `verify()` расположено перед определением функции `setHeight()`, а потому может использоваться.

Упражнения раздела 7.4.1

Упражнение 7.34. Что произойдет, если поместить определение типа `pos` в последнюю строку класса `Screen` на стр. 370?

Упражнение 7.35. Объясните код, приведенный ниже. Укажите, какое из определений, `Type` или `initVal`, будет использовано для каждого из имен. Если здесь есть ошибки, найдите и исправьте их.

```
typedef string Type;
Type initVal();
class Exercise {
public:
    typedef double Type;
    Type setVal(Type);
    Type initVal();
private:
    int val;
};
Type Exercise::setVal(Type parm) {
    val = parm + initVal();
    return val;
}
```

7.5. Снова о конструкторах

Конструкторы — ключевая часть любого класса C++. Основы конструкторов рассматривались в разделе 7.1.4 (стр. 342), а в этом разделе описаны некоторые из дополнительных возможностей конструкторов и подробности материала, приведенного ранее.



7.5.1. Список инициализации конструктора

Когда определяются переменные, они, как правило, инициализируются сразу, а не определяются и присваиваются впоследствии:

```
string foo = "Hello World!"; // определить и инициализировать
string bar;                  // инициализация по умолчанию пустой строкой
bar = "Hello World!";       // присвоение нового значения переменной bar
```

Аналогичное различие между инициализацией и присвоением относится к переменным-членам объектов. Если не инициализировать переменную-член явно в списке инициализации конструктора, она инициализируется значением по умолчанию прежде, чем выполнится тело конструктора. Например:

```
// допустимый, но не самый лучший способ создания конструктора
// класса Sales_data: нет инициализатора конструктора
Sales_data::Sales_data(const string &s,
                      unsigned cnt, double price)
{
    bookNo = s;
    units_sold = cnt;
    revenue = cnt * price;
}
```

Эта версия и исходное определение на стр. 345 дают одинаковый результат: по завершении конструктора переменные-члены содержат те же значения. Различие в том, что исходная версия *инициализирует* свои переменные-члены, тогда как эта версия *присваивает* значения им. Насколько существенно это различие, зависит от типа переменной-члена.

Иногда применение списка инициализации конструктора неизбежно

Зачастую, *но не всегда*, можно игнорировать различие между инициализацией и присвоением значения переменной-члену. Переменные-члены, являющиеся константой или ссылкой, должны быть инициализированы. Аналогично члены класса, для типа которых не определен стандартный конструктор, также следует инициализировать. Например:

```
class ConstRef {
public:
    ConstRef(int ii);
private:
    int i;
    const int ci;
    int &ri;
};
```

Переменные-члены `ci` и `ri` следует инициализировать как любой другой константный объект или ссылку. В результате отсутствие инициализатора конструктора для этих членов будет ошибкой:

```
// ошибка: ci и ri должны быть инициализированы
ConstRef::ConstRef(int ii)
{ // присвоения:
    i = ii; // ok
```

```

ci = ii; // ошибка: нельзя присвоить значение константе
ri = i;  // ошибка: ri никогда не будет инициализирована
}

```

К тому времени, когда начинает выполняться тело конструктора, инициализация уже завершена. Единственный шанс инициализировать константу или ссылочную переменную-член — в инициализаторе конструктора. Вот правильный способ написания этого конструктора:

```

// ok: явная инициализация констант и ссылок
ConstRef::ConstRef(int ii): i(ii), ci(ii), ri(i) { }

```



Для предоставления значений переменным-членам, являющимся константой, ссылкой или классом, у которого нет стандартного конструктора, использование списка инициализации конструктора неизбежно.

СОВЕТ. ИСПОЛЬЗУЙТЕ СПИСКИ ИНИЦИАЛИЗАЦИИ КОНСТРУКТОРА

Во многих классах различие между инициализацией и присвоением связано исключительно с вопросом эффективности: зачем инициализировать переменную-член и присваивать ей значение, когда ее достаточно просто инициализировать.

Однако важнее эффективности тот факт, что некоторые переменные-члены обязательно должны быть инициализированы. При стандартном использовании инициализаторов конструктора можно избежать неожиданных ошибок компиляции, когда класс обладает членом, требующим наличия списка инициализации.

Порядок инициализации переменных-членов класса

Нет ничего удивительного в том, что каждая переменная-член присутствует в списке инициализации конструктора только один раз. В конце концов, зачем переменной-члену два исходных значения?

Но что на самом деле неожиданно, так это то, что список инициализации конструктора задает только значения, используемые для инициализации переменных-членов, но не определяет порядок, в котором осуществляется инициализация.

Порядок инициализации переменных-членов задает их расположение при определении. Порядок расположения инициализаторов в списке инициализации конструктора не влияет на порядок инициализации.

Порядок инициализации зачастую не имеет значения. Но если одна из переменных-членов инициализируется с учетом значения другой, порядок их инициализации критически важен.

В качестве примера рассмотрим следующий класс:


```
class X {
    int i;
    int j;
public:
    // ошибка: i инициализируется прежде j
    X(int val): j(val), i(j) { }
};
```

В данном случае список инициализации конструктора написан так, чтобы инициализировать переменную-член `j` значением `val`, а затем использовать переменную-член `j` для инициализации переменной-члена `i`. Но переменная-член `i` инициализируется первой. В результате попытка инициализации переменной-члена `i` осуществляется в момент, когда переменная-член `j` еще не имеет значения!

Некоторые компиляторы достаточно интеллектуальны, чтобы распознать опасность и выдать предупреждение о том, что переменные-члены в списке инициализации конструктора расположены в порядке, отличном от порядка их объявления.



Элементы списка инициализации конструктора имеет смысл располагать в том же порядке, в котором объявлены переменные-члены. Кроме того, старайтесь по возможности избегать применения одних переменных-членов для инициализации других.

Вообще, можно достаточно просто избежать любых проблем, связанных с порядком выполнения инициализации. Достаточно использовать параметры конструктора вместо переменных-членов объекта. Конструктор класса `X`, например, лучше было бы написать следующим образом:

```
X(int val): i(val), j(val) { }
```

В этой версии порядок инициализации переменных-членов `i` и `j` не имеет значения.

Аргументы по умолчанию и конструкторы

Действие стандартного конструктора класса `Sales_data` подобно конструктору, получающему один строковый аргумент. Единственное отличие в том, что конструктор, получающий строковый аргумент, использует его для инициализации переменной-члена `bookNo`. Стандартный конструктор (неявно) использует стандартный конструктор типа `string` для инициализации переменной `bookNo`. Эти конструкторы можно переписать как единый конструктор с аргументом по умолчанию (см. раздел 6.5.1, стр. 308):

```
class Sales_data {
public:
    // определить стандартный конструктор как получающий строковый
    // аргумент
```

```

Sales_data(std::string s = ""): bookNo(s) { }
// остальные конструкторы без изменений
Sales_data(std::string s, unsigned cnt, double rev):
    bookNo(s), units_sold(cnt), revenue(rev*cnt) { }
Sales_data(std::istream &is) { read(is, *this); }
// остальные члены, как прежде
};

```

Эта версия класса предоставляет тот же интерфейс, что и исходный на стр. 345. Обе версии создают тот же объект, когда никаких аргументов не предоставлено или когда предоставлен один строковый аргумент. Поскольку этот конструктор можно вызвать без аргументов, он считается стандартным конструктором класса.



Конструктор, предоставляющий аргументы по умолчанию для всех своих параметров, также считается стандартным конструктором.

Следует заметить, что, вероятно, не нужно использовать аргументы по умолчанию с конструктором `Sales_data()`, который получает три аргумента. Если пользователь предоставляет не нулевое количество проданных книг, следует также гарантировать, что пользователь предоставит и цену, по которой они были проданы.

Упражнения раздела 7.5.1

Упражнение 7.36. Следующий инициализатор ошибочен. Найдите и исправьте ошибку.

```

struct X {
    X (int i, int j): base(i), rem(base % j) { }
    int rem, base;
};

```

Упражнение 7.37. Используя версию класса `Sales_data` из этого раздела, определите, какой конструктор используется для инициализации каждой из следующих переменных, а также перечислите значения переменных-членов в каждом объекте:

```

Sales_data first_item(cin);

int main() {
    Sales_data next;
    Sales_data last("9-999-99999-9");
}

```

Упражнение 7.38. Конструктору, получающему аргумент типа `istream&`, можно предоставить объект `cin` как аргумент по умолчанию. Напишите объявление конструктора, использующего объект `cin` как аргумент по умолчанию.

Упражнение 7.39. Допустимо ли для конструктора, получающего строку, и конструктора, получающего тип `istream&`, иметь аргументы по умолчанию? Если нет, то почему?

Упражнение 7.40. Выберите одну из следующих абстракций (или абстракцию по собственному выбору). Определите, какие данные необходимы в классе. Предоставьте соответствующий набор конструкторов. Объясните свои решения.

- | | | |
|-------------|------------|--------------|
| (a) Book | (b) Date | (c) Employee |
| (d) Vehicle | (e) Object | (f) Tree |

7.5.2. Делегирующий конструктор

**C++
11**

Новый стандарт расширяет использование списков инициализации конструктора, позволяя определять так называемые *делегирующие конструкторы* (delegating constructor). Делегирующий конструктор использует для инициализации другой конструктор своего класса. Он “делегирует” некоторые (или все) свои задачи другому конструктору.

Подобно любому другому конструктору, делегирующий конструктор имеет список инициализации переменных-членов и тело функции. Список инициализации делегирующего конструктора содержит элемент, являющийся именем самого класса. Подобно другим инициализаторам переменных-членов класса, имя класса сопровождается заключенным в скобки списком аргументов. Список аргументов должен соответствовать другому конструктору в классе.

В качестве примера перепишем класс `Sales_data` так, чтобы использовать делегирующие конструкторы следующим образом:

```
class Sales_data {
public:
    // недеlegeирующий конструктор инициализирует члены из соответствующих
    // аргументов
    Sales_data(std::string s, unsigned cnt, double price):
        bookNo(s), units_sold(cnt), revenue(cnt*price) { }
    // все другие конструкторы делегируют к другому конструктору
    Sales_data(): Sales_data("", 0, 0) {}
    Sales_data(std::string s): Sales_data(s, 0, 0) {}
    Sales_data(std::istream &is): Sales_data()
        { read(is, *this); }
    // другие члены как прежде
}
```

В этой версии класса `Sales_data` все конструкторы, кроме одного, делегируют свою работу. Первый конструктор получает три аргумента и использует их для инициализации переменных-членов, но ничего другого не делает. В этой версии класса определен стандартный конструктор, использующий для инициализации конструктор на три аргумента. Он также не делает ничего,

поэтому его тело пусто. Конструктор, получающий строку, также делегирует работу версии на три аргумента.

Конструктор, получающий объект `istream&`, также делегирует свои действия. Он делегирует их стандартному конструктору, который в свою очередь делегирует их конструктору на три аргумента. Как только эти конструкторы заканчивают свою работу, запускается тело конструктора с аргументом `istream&`. Оно вызывает функцию `read()` для чтения данных из потока `istream`.

Когда конструктор делегирует работу другому конструктору, список инициализации и тело делегированного конструктора выполняются оба. В классе `Sales_data` тела делегируемых конструкторов пусты. Если бы тела конструкторов содержали код, то он выполнялся бы прежде, чем управление возвратилось бы к телу делегирующего конструктора.

Упражнения раздела 7.5.2

Упражнение 7.41. Перепишите собственную версию класса `Sales_data`, чтобы использовать делегирующие конструкторы. Добавьте в тело каждого конструктора оператор, выводящий сообщение всякий раз, когда он выполняется. Напишите объявления для создания объекта класса `Sales_data` любыми возможными способами. Изучите вывод и удостоверьтесь, что понимаете порядок выполнения делегирующих конструкторов.

Упражнение 7.42. Вернитесь к классу, написанному для упражнения 7.40 в разделе 7.5.1 (стр. 377), и решите, может ли какой-нибудь из его конструкторов использовать делегирование. Если да, то напишите делегирующий конструктор (конструкторы) для своего класса. В противном случае рассмотрите список абстракций и выберите ту, которая, по вашему, использовала бы делегирующий конструктор. Напишите определение класса для этой абстракции.



7.5.3. Роль стандартного конструктора

Стандартный конструктор автоматически используется всякий раз, когда объект инициализируется по умолчанию. Инициализация по умолчанию осуществляется в следующем случае.

- При определении нестатических переменных (см. раздел 2.2.1, стр. 77) или массивов (см. раздел 3.5.1, стр. 162) в области видимости блока без инициализаторов.
- Когда класс, который сам обладает членами типа класса, использует синтезируемый стандартный конструктор (см. раздел 7.1.4, стр. 342).
- Когда переменные-члены типа класса не инициализируются явно в списке инициализации конструктора (см. раздел 7.1.4, стр. 345).

Инициализация значением по умолчанию осуществляется в следующем случае.

- Во время инициализации массива, когда предоставляется меньше инициализаторов, чем элементов массива (см. раздел 3.5.1, стр. 162).
- При определении локального статического объекта без инициализатора (см. раздел 6.1.1, стр. 272).
- Когда явно запрашивается инициализация значением по умолчанию в форме выражения `T()`, где `T` — это имя типа. (Конструктор вектора, получающий один аргумент, чтобы определить размер вектора (см. раздел 3.3.1, стр. 143), использует аргумент этого вида для инициализации значением по умолчанию своего элемента.)

Чтобы использоваться в этих контекстах, у классов должен быть стандартный конструктор. Большинство этих контекстов должно быть вполне очевидным.

Однако значительно менее очевидным может быть влияние на классы, у которых есть переменные-члены без стандартного конструктора:

```
class NoDefault {
public:
    NoDefault(const std::string&);
    // далее дополнительные члены, но нет других конструкторов
};
struct A { // my_mem является открытой по умолчанию; см. раздел 7.2,
    // стр. 350
    NoDefault my_mem;
};
A a; // ошибка: невозможен синтезируемый конструктор для A
struct B {
    B() {} // ошибка: нет инициализатора для b_member
    NoDefault b_member;
};
```

Рекомендуем На практике почти всегда имеет смысл предоставлять стандартный конструктор, если определены другие конструкторы.

Применение стандартного конструктора

Следующее объявление объекта `obj` компилируется без проблем. Но при попытке его использования компилятор жалуется на невозможность применения к функции синтаксиса доступа к члену.

```
Sales_data obj(); // ok: не определена функция, а не объект
if (obj.isbn() == Primer_5th_ed.isbn()) // ошибка: obj - функция
```

Проблема в том, что, несмотря на намерение объявить инициализированный значением по умолчанию объект `obj`, фактически была объявлена функция без параметров, возвращающая объект типа `Sales_data`.

Чтобы правильно определить объект, использующий стандартный конструктор для инициализации, следует убрать пустые круглые скобки:

```
// ok: obj - объект, инициализированный значением по умолчанию
Sales_data obj;
```



ВНИМАНИЕ

Распространенной ошибкой среди новичков в C++ является объявление объекта, инициализированного стандартным конструктором, следующим образом:

```
Sales_data obj(); // ups! Это объявление функции, а не создание объекта
Sales_data obj2; // ok: obj2 - это объект, а не функция
```

Упражнения раздела 7.5.3

Упражнение 7.43. Предположим, имеется класс `NoDefault`, у которого есть конструктор, получающий параметр типа `int`, но нет стандартного конструктора. Определите класс `C`, у которого есть переменная-член типа `NoDefault`. Определите стандартный конструктор для класса `C`.

Упражнение 7.44. Допустимо ли следующее объявление? Если нет, то почему?

```
vector<NoDefault> vec(10);
```

Упражнение 7.45. Определите вектор, содержащий объекты типа `C` из предыдущего упражнения?

Упражнение 7.46. Которое из следующих утверждений, если таковое имеется, ложно? Почему?

- (a) Класс должен предоставить по крайней мере один конструктор.
- (b) Стандартный конструктор — это конструктор с пустым списком параметров.
- (c) Если для класса не нужно никаких значений по умолчанию, то класс не должен предоставлять стандартный конструктор.
- (d) Если класс не определяет стандартный конструктор, компилятор сам создает конструктор, который инициализирует каждую переменную-член значением по умолчанию соответствующего типа.



7.5.4. Неявное преобразование типов класса

Как упоминалось в разделе 4.11 (стр. 218), язык C++ автоматически осуществляет преобразование некоторых встроенных типов. Обращалось также внимание на то, что классы тоже могут определять неявные преобразования. Каждый конструктор, который может быть вызван с одним аргументом, определяет неявное преобразование в тип класса. Такие конструкторы иногда

упоминают как *конструкторы преобразования* (converting constructor). Определение преобразования из типа класса в другой тип рассматривается в разделе 14.9 (стр. 732).



Конструктор, который может быть вызван с одиночным аргументом, вполне позволяет определить неявное преобразование из типа параметра в тип класса.

Конструкторы класса `Sales_data`, получающие строку и объект класса `istream`, оба определяют неявные преобразования из этих типов в тип `Sales_data`. Таким образом, можно использовать тип `string` или `istream` там, где ожидается объект типа `Sales_data`:

```
string null_book = "9-999-99999-9";
// создает временный объект типа Sales_data
// с units_sold и revenue равными 0 и bookNo равным null_book
item.combine(null_book);
```

Здесь происходит вызов функции-члена `combine()` класса `Sales_data` со строковым аргументом. Этот вызов совершенно корректен; компилятор автоматически создаст объект класса `Sales_data` из данной строки. Этот вновь созданный (временный) объект класса `Sales_data` передается функции `combine()`. Поскольку параметр функции `combine()` является ссылкой на константу, этому параметру можно передать временный объект.

Допустимо только одно преобразование типов класса

В разделе 4.11.2 (стр. 222) обращалось внимание на то, что компилятор автоматически применит только одно преобразование типов класса. Например, следующий код ошибочен, поскольку он неявно использует два преобразования:

```
// ошибка: требует двух пользовательских преобразований:
//      (1) преобразование "9-999-99999-9" в string
//      (2) преобразование временной строки в Sales_data
item.combine("9-999-99999-9");
```

Если данный вызов необходим, это можно сделать при явном преобразовании символьной строки в объект класса `string` или в объект класса `Sales_data`:

```
// ok: явное преобразование в string, неявное преобразование в Sales_data
item.combine(string("9-999-99999-9"));
// ok: неявное преобразование в string, явное преобразование в Sales_data
item.combine(Sales_data("9-999-99999-9"));
```

Преобразования типов класса не всегда полезны

Желательно ли преобразование типа `string` в `Sales_data`, зависит от конкретных обстоятельств. В данном случае это хорошая идея. Строка в переменной `null_book`, вероятнее всего, соответствует несуществующему ISBN.

Преобразование из `istream` в `Sales_data` более проблематично:

```
// использует конструктор istream при создании объекта для передачи
// функции combine
item.combine(cin);
```

Этот код неявно преобразует объект `cin` в объект класса `Sales_item`. Это преобразование осуществляет тот конструктор класса `Sales_data`, который получает тип `istream`. Этот конструктор создает (временный) объект класса `Sales_data` при чтении со стандартного устройства ввода. Затем этот объект передается функции `same_isbn()`.

Этот объект класса `Sales_item` временный (см. раздел 2.4.1, стр. 99). По завершении функции `combine()` никакого доступа к нему не будет. Фактически создается объект, удаляющийся после того, как его значение добавляется в объект `item`.

Предотвращение неявных преобразований, осуществляемых конструктором

Чтобы предотвратить использование конструктора в контексте, который требует неявного преобразования, достаточно объявить его *явным* (`explicit constructor`) с использованием ключевого слова `explicit`:

```
class Sales_data {
public:
    Sales_data() = default;
    Sales_data(const std::string &s, unsigned n, double p):
        bookNo(s), units_sold(n), revenue(p*n) { }
    explicit Sales_data(const std::string &s): bookNo(s) { }
    explicit Sales_data(std::istream&);
    // остальные члены, как прежде
};
```

Теперь ни один из конструкторов не применим для неявного создания объектов класса `Sales_data`. Ни один из предыдущих способов применения теперь не сработает:

```
item.combine(null_book); // ошибка: конструктор string теперь явный
item.combine(cin);       // ошибка: конструктор istream теперь явный
```

Ключевое слово `explicit` имеет значение только для тех конструкторов, которые могут быть вызваны с одним аргументом. Конструкторы, требующие большего количества аргументов, не используются для неявного преобразования, поэтому нет никакой необходимости определять их как `explicit`. Ключевое слово `explicit` используется только в объявлениях конструкторов в классе. В определении вне тела класса его не повторяют.

```
// ошибка: ключевое слово explicit допустимо только для
// объявлений конструкторов в заголовке класса
explicit Sales_data::Sales_data(istream& is)
{
    read(is, *this);
}
```


Явные конструкторы применяются только для прямой инициализации

Одним из контекстов, в котором происходит неявное преобразования, является использование формы инициализации копированием (со знаком =) (см. раздел 3.2.1, стр. 126). С этой формой инициализации нельзя использовать явный конструктор; придется использовать прямую инициализацию:

```
Sales_data item1(null_book); // ok: прямая инициализация
// ошибка: с явным конструктором нельзя использовать форму
// инициализации копированием
Sales_data item2 = null_book;
```



Явный конструктор применим только с прямой формой инициализации (см. раздел 3.2.1, стр. 126). Кроме того, компилятор *не будет* использовать этот конструктор в автоматическом преобразовании.

Применение явных конструкторов для преобразований

Хотя компилятор не будет использовать явный конструктор для неявного преобразования, его можно использовать для преобразования явно:

```
// ok: аргумент - явно созданный объект класса Sales_data
item.combine(Sales_data(null_book));
// ok: static_cast может использовать явный конструктор
item.combine(static_cast<Sales_data>(cin));
```

В первом вызове конструктор `Sales_data()` используется непосредственно. Этот вызов создает временный объект класса `Sales_data`, используя конструктор `Sales_data()`, получающий строку. Во втором вызове используется оператор `static_cast` (см. раздел 4.11.3, стр. 223) для выполнения явного, а не неявного преобразования. В этом вызове оператор `static_cast` использует для создания временного объекта класса `Sales_data` конструктор с параметром типа `istream`.

Библиотечные классы с явными конструкторами

У некоторых библиотечных классов, включая уже использованные ранее, есть конструкторы с одним параметром.

- Конструктор класса `string`, получающий один параметр типа `const char*` (см. раздел 3.2.1, стр. 126), не является явным.
- Конструктор класса `vector`, получающий размер вектора (см. раздел 3.3.1, стр. 143), является явным.

Упражнения раздела 7.5.4

Упражнение 7.47. Объясните, должен ли быть явным конструктор `Sales_data()`, получающий строку. Каковы преимущества объявления конструктора явным? Каковы недостатки?

Упражнение 7.48. С учетом того, что конструктор `Sales_data()` не является явным, какие операции происходят во время следующих определений:

```
string null_isbn("9-999-99999-9");
Sales_data item1(null_isbn);
Sales_data item2("9-999-99999-9");
```

Что будет при явном конструкторе `Sales_data()`?

Упражнение 7.49. Объясните по каждому из следующих трех объявлений функции `combine()`, что происходит при вызове `i.combine(s)`, где `i` — это объект класса `Sales_data`, а `s` — строка:

- (a) `Sales_data &combine(Sales_data);`
- (b) `Sales_data &combine(Sales_data&);`
- (c) `Sales_data &combine(const Sales_data&) const;`

Упражнение 7.50. Определите, должен ли какой-либо из конструкторов вашего класса `Person` быть явным.

Упражнение 7.51. Как, по вашему, почему вектор определяет свой конструктор с одним аргументом как явный, а строка нет?



7.5.5. Агрегатные классы

Агрегатный класс (aggregate class) предоставляет пользователям прямой доступ к своим членам и имеет специальный синтаксис инициализации. Класс считается агрегатным в следующем случае.

- Все его переменные-члены являются открытыми (`public`).
- Он не определяет конструкторы.
- У него нет никаких внутриклассовых инициализаторов (см. раздел 2.6.1, стр. 113).
- У него нет никаких базовых классов или виртуальных функций, связанных с классом средствами, которые рассматриваются в главе 15.

Например, следующий класс является агрегатным:

```
struct Data {
    int ival;
    string s;
};
```

Для инициализации переменных-членов агрегатного класса можно предоставить заключенный в фигурные скобки список инициализаторов для переменных-членов:

```
// val1.ival = 0; val1.s = string("Anna")
Data val1 = { 0, "Anna" };
```

Инициализаторы должны располагаться в порядке объявления переменных-членов. Таким образом, сначала располагается инициализатор для первой переменной-члена, затем для второй и т.д. Следующей пример ошибочен:

```
// ошибка: нельзя использовать "Anna" для инициализации ival или 1024
// для инициализации s
Data val2 = { "Anna" , 1024 };
```

Как и при инициализации элементов массива (см. раздел 3.5.1, стр. 162), если в списке инициализаторов меньше элементов, чем переменных-членов класса, последние переменные-члены инициализируются значением по умолчанию. Список инициализаторов не должен содержать больше элементов, чем переменных-членов у класса.

Следует заметить, что у явной инициализации переменных-членов объекта класса есть три существенных недостатка.

- Она требует, чтобы все переменные-члены были открытыми.
- Налагает дополнительные обязанности по правильной инициализации каждой переменной-члена каждого объекта на пользователя класса (а не на его автора). Такая инициализация утомительна и часто приводит к ошибкам, поскольку достаточно просто забыть инициализатор или предоставить неподходящее значение.
- Если добавляется или удаляется переменная-член, придется изменить все случаи инициализации.

Упражнения раздела 7.5.5

Упражнение 7.52. На примере первой версии класса `Sales_data` из раздела 2.6.1 (стр. 112) объясните следующую инициализацию. Найдите и исправьте возможные ошибки.

```
Sales_data item = {"978-0590353403", 25, 15.99};
```



7.5.6. Литеральные классы

В разделе 6.5.2 (стр. 312) упоминалось, что параметры и возвращаемое значение функции `constexpr` должны иметь литеральные типы. Кроме арифметических типов, ссылок и указателей, некоторые классы также являются литеральными типами. В отличие от других классов, у классов, являющихся лите-

ральными типами, могут быть функции-члены constexpr. Такие функции-члены должны отвечать всем требованиям функций constexpr. Эти функции-члены неявно константные (см. раздел 7.1.2, стр. 337).

Агрегатный класс (см. раздел 7.5.5, стр. 384), все переменные-члены которого имеют литеральный тип, является литеральным классом. Неагрегатный класс, соответствующий следующим ограничениям, также является литеральным классом.

- У всех переменных-членов должен быть литеральный тип.
- У класса должен быть по крайней мере один конструктор constexpr.
- Если у переменной-члена есть внутриклассовый инициализатор, инициализатор для переменной-члена встроенного типа должен быть константным выражением (см. раздел 2.4.4, стр. 103). Если переменная-член имеет тип класса, инициализатор должен использовать его собственный конструктор constexpr.
- Класс должен использовать заданное по умолчанию определение для своего деструктора — функции-члена класса, удаляющего объекты типа класса (см. раздел 7.1.5, стр. 347).

Конструкторы constexpr

Хотя конструкторы не могут быть константными (см. раздел 7.1.4, стр. 342), в литеральном классе они могут быть функциями constexpr (см. раздел 6.5.2, стр. 312). Действительно, литеральный класс должен предоставлять по крайней мере один конструктор constexpr.

**C++
11** Конструктор constexpr может быть объявлен как = default (см. раздел 7.1.4, стр. 345) или как удаленная функция, которые будут описаны в разделе 13.1.6 (стр. 643). В противном случае конструктор constexpr должен отвечать требованиям к конструкторам (у него не может быть оператора return) и к функциям constexpr (его исполняемый оператор может иметь единственный оператор return) (см. раздел 6.5.2, стр. 312). В результате тело конструктора constexpr обычно пусто. Определению конструктора constexpr предшествует ключевое слово constexpr:

```
class Debug {
public:
    constexpr Debug(bool b = true): hw(b), io(b), other(b) { }
    constexpr Debug(bool h, bool i, bool o):
        hw(h), io(i), other(o) { }
    constexpr bool any() { return hw || io || other; }
    void set_io(bool b) { io = b; }
    void set_hw(bool b) { hw = b; }
    void set_other(bool b) { hw = b; }
private:
    bool hw;    // аппаратная ошибка, отличная от ошибки IO
```

```

    bool io;      // ошибка IO
    bool other;   // другие ошибки
};

```

Конструктор `constexpr` должен инициализировать каждую переменную-член. Инициализаторы должны либо использовать конструктор `constexpr`, либо быть константным выражением.

Конструктор `constexpr` используется и для создания объектов `constexpr`, и для параметров или типов возвращаемого значения функций `constexpr`:

```

constexpr Debug io_sub(false, true, false); // отладка IO
if (io_sub.any())                          // эквивалент if(true)
    cerr << "print appropriate error messages" << endl;
constexpr Debug prod(false);               // при выпуске без отладки
if (prod.any())                            // эквивалент if(false)
    cerr << "print an error message" << endl;

```

Упражнения раздела 7.5.6

Упражнение 7.53. Определите собственную версию класса `Debug`.

Упражнение 7.54. Должны ли члены класса `Debug`, начинающиеся с `set_`, быть объявлены как `constexpr`? Если нет, то почему?

Упражнение 7.55. Является ли класс `Data` из раздела 7.5.5 (стр. 384) литеральным? Если нет, то почему? Если да, то почему он является литеральным.

7.6. Статические члены класса

Иногда классы нуждаются в членах, ассоциированных с самим классом, а не с его индивидуальными объектами. Например, класс банковского счета, возможно, нуждается в переменной-члене, представляющей базовую процентную ставку. В данном случае мы хотели бы ассоциировать процентную ставку с классом, а не с каждым конкретным объектом. С точки зрения эффективности нет никаких причин хранить процентную ставку для каждого объекта. Однако важнее всего то, что если процентная ставка изменится, каждый объект сразу использует новое значение.

Объявление статических членов

Чтобы сделать член класса статическим, его объявление следует предварить ключевым словом `static`. Статические члены, как и любые другие, могут быть открытыми или закрытыми. Статическая переменная-член может быть константой, ссылкой, массивом, классом и т.д.

В качестве примера определим класс, представляющий банковскую учетную запись:

```

class Account {
public:
    void calculate() { amount += amount * interestRate; }
    static double rate() { return interestRate; }
    static void rate(double);
private:
    std::string owner;
    double amount;
    static double interestRate;
    static double initRate();
};

```

Статические члены класса существуют вне конкретного объекта. Объекты не содержат данные, связанные со статическими переменными-членами. Таким образом, каждый объект класса `Account` будет содержать две переменные-члена — `owner` и `amount`. Есть только один объект `interestRate`, совместно используемый всеми объектами `Account`.

Аналогично статические функции-члены не связаны с конкретным объектом; у них нет указателя `this`. В результате статические функции-члены не могут быть объявлены константами и к указателю `this` нельзя обратиться в теле статического члена класса. Это ограничение применимо и к явному использованию указателя `this`, и к неявному, при вызове не статического члена класса.

Использование статических членов класса

К статическому члену класса можно обратиться непосредственно, используя оператор области видимости:

```

double r;
r = Account::rate(); // доступ к статическому члену при помощи оператора
                     // области видимости

```

Даже при том, что статические члены не являются частью отдельных объектов, для доступа к статическому члену класса можно использовать объект, ссылку или указатель на тип класса:

```

Account ac1;
Account *ac2 = &ac1;
// эквивалентные способы вызова статической функции rate
r = ac1.rate(); // через объект класса Account или ссылку
r = ac2->rate(); // через указатель на объект класса Account

```

Функции-члены могут использовать статические члены непосредственно, без оператора области видимости:

```

class Account {
public:
    void calculate() { amount += amount * interestRate; }
private:
    static double interestRate;
    // остальные члены как прежде
};

```

Определение статических членов

Подобно любой другой функции-члену, статическую функцию-член можно определить как в, так и вне тела класса. Когда статический член класса определяется вне его тела класса, ключевое слово `static` повторять не нужно, оно присутствует только в объявлении в теле класса:

```
void Account::rate(double newRate)
{
    interestRate = newRate;
}
```



При обращении к статическому члену класса вне тела класса, подобно любому другому члену класса, необходимо указать класс, в котором он определен. Но ключевое слово `static` используется *только* при объявлении в теле класса. В определении ключевое слово `static` не используется.

Поскольку статические переменные-члены на принадлежат индивидуальным объектам класса, они не создаются при создании объектов класса. В результате они не инициализируются конструкторами класса. Кроме того, статическую переменную-член вообще нельзя инициализировать в классе. Каждую статическую переменную-член следует определить и инициализировать вне тела класса. Как и любой другой объект, статическая переменная-член может быть определена только однажды.

Как и глобальные объекты (см. раздел 6.1.1, стр. 271), статические переменные-члены определяются вне любой функции. Следовательно, сразу после определения они продолжают существовать, пока программа не завершит работу.

Статические члены определяют точно так же, как и функции-члены класса вне класса. Указывается тип объекта, затем имя класса, оператор области видимости и собственное имя члена:

```
// определить и инициализировать статический член класса
double Account::interestRate = initRate();
```

Этот оператор определяет статический объект по имени `interestRate`, который является членом класса `Account` и имеет тип `double`. Подобно другим членам класса, определение статического находится в области видимости того класса, где определено его имя. В результате статическую функцию-член `initRate()` можно использовать для инициализации переменной `rate` непосредственно, без уточнения класса. Обратите внимание: несмотря на то, что функция-член `initRate()` является закрытой, ее можно использовать для инициализации объекта `interestRate`. Определение переменной-члена `interestRate`, подобно любому другому определению, находится в области видимости класса, а следовательно, имеет доступ к закрытым членам класса.



Наилучший способ гарантировать, что объект будет определен только один раз, — разместить определение статических переменных-членов в том же файле, который содержит определение не встраиваемых функций-членов класса.

Инициализация статических переменных-членов в классе

Обычно статические переменные-члены не могут быть инициализированы в теле класса. Но можно предоставить внутриклассовые инициализаторы для тех статических переменных-членов, которые имеют тип целочисленных констант, или статических членов `constexpr` литерального типа (см. раздел 7.5.6, стр. 385). Инициализаторы должны быть константными выражениями. Такие члены сами являются константными выражениями; они могут быть использованы там, где ожидается константное выражение. Например, инициализированную статическую переменную-член можно использовать для определения размерности члена типа массива:

```
class Account {
public:
    static double rate() { return interestRate; }
    static void rate(double);
private:
    static constexpr int period = 30; // period - константное выражение
    double daily_tbl[period];
};
```

Если член класса используется только в контекстах, где компилятор может подставить его значение, то инициализированная константа или статическое константное выражение не следует определять отдельно. Но если член класса используется в контексте, где значение не может быть подставлено, то определение для этого члена необходимо.

Например, если переменная `period` используется только для определения размерности массива `daily_tbl`, нет никакой необходимости определять ее за пределами класса `Account`. Но если пропустить определение, то даже, казалось бы, тривиальное изменение в программе может привести к отказу компиляции. Например, если передать переменную-член `Account::period` функции, получающей параметр типа `const int&`, то переменную `period` следует определить.

Если инициализатор предоставляется в классе, определение члена класса не должно задавать исходного значения:

```
// определение статического члена без инициализатора
constexpr int Account::period; // инициализатор предоставлен в
                               // определении класса
```




Даже если константная статическая переменная-член инициализируется в теле класса, она должна определяться вне определения класса.

Статические члены можно применять так, как нельзя применять обычные

Как уже упоминалось, статические члены существуют независимо от конкретного объекта. В результате они применимы такими способами, которые недопустимы для нестатических переменных-членов. Например, у статической переменной-члена может быть незавершенный тип (см. раздел 7.3.3, стр. 362). В частности, статическая переменная-член может иметь тип, совпадающий с типом класса, членом которого она является. Нестатическая переменная-член может быть только указателем или ссылкой на объект собственного класса:

```
class Bar {
public:
    // ...
private:
    static Bar mem1; // ok: тип статического члена может быть
                    // незавершенным
    Bar *mem2;       // ok: тип указателя-члена может быть незавершенным
    Bar mem3;        // ошибка: тип переменной-члена должен быть
                    // завершенным
};
```

Еще одно различие между статическими и обычными членами в том, что статический член можно использовать как аргумент по умолчанию (см. раздел 6.5.1, стр. 308):

```
class Screen {
public:
    // bkground ссылается на статический член класса
    // объявлено позже, в определении класса
    Screen& clear(char = bkground);
private:
    static const char bkground;
};
```

Нестатическая переменная-член не может использоваться как аргумент по умолчанию, поскольку ее значение является частью объекта, которому она принадлежит. Использование нестатической переменной-члена как аргумента, по умолчанию не предоставляющего объект, которому она принадлежит, также является ошибкой.

Упражнения раздела 7.6

Упражнение 7.56. Что такое статический член класса? Каковы преимущества статических членов? Чем они отличаются от обычных членов?

Упражнение 7.57. Напишите собственную версию класса `Account`.

Упражнение 7.58. Какие из следующих объявлений и определений статических переменных-членов являются ошибочными? Объясните почему.

```
// example.h
class Example {
public:
    static double rate = 6.5;
    static const int vecSize = 20;
    static vector<double> vec(vecSize);
};
// example.C
#include "example.h"
double Example::rate;
vector<double> Example::vec;
```

Резюме

Классы — это фундаментальный компонент языка C++. Классы позволяют определять новые типы, наилучшим образом приспособленные к задачам конкретного приложения и позволяющие сделать их короче и проще в модификации.

Основой классов являются абстракция данных (способность определять данные и функции-члены) и инкапсуляция (способность защитить члены класса от общего доступа). Инкапсуляция класса достигается определением членов его реализации закрытыми. Классы могут предоставить доступ к своему не открытому члену, объявив другой класс или функцию дружественной.

Классы могут определять конструкторы — специальные функции-члены, контролирующие инициализацию объектов. Конструкторы могут быть перегружены. Для инициализации всех переменных-членов конструкторы должны использовать список инициализации конструктора.

Классы позволяют объявлять переменные-члены изменяемыми (`mutable`) или статическими (`static`). Изменяемая переменная-член никогда не становится константой — ее значение может быть изменено даже в константной функции-члене. Статической может быть как функция, так и переменная-член. Статические члены класса существуют независимо от объектов данного класса.

Классы могут также определить изменяемые (`mutable`) и статические (`static`) члены. Изменяемая переменная-член никогда не становится константой; ее значение может быть изменено даже в константной функции-члене. Статический член может быть функцией или переменной; статические члены существуют независимо от объектов типа класса.

Термины

= default. Синтаксис, используемый после списка параметров объявления стандартного конструктора класса, чтобы сообщить компилятору о необходимости создать конструктор, даже если у класса есть другие конструкторы.

Абстрактный тип данных (abstract data type). Структура данных, инкапсулирующая (скрывающая) свою реализацию.

Абстракция данных (data abstraction). Технология программирования, сосредоточенная на интерфейсе типа. Абстракция данных позволяет программистам игнорировать детали реализации типа, интересуясь лишь его возможностями. Абстракция данных является основой как объектно-ориентированного, так и обобщенного программирования.

Агрегатный класс (aggregate class). Класс только с открытыми переменными-членами, без внутриклассовых инициализаторов или конструкторов. Члены агрегатного класса могут быть инициализированы заключенным в фигурные скобки списком инициализаторов.

Делегирующий конструктор (delegating constructor). Конструктор со списком инициализации, один элемент которого определяет другой конструктор того же класса для инициализации.

Дружественные отношения (friend). Механизм, при помощи которого класс предоставляет доступ к своим не открытым членам. Дружественные классы и функции имеют те же права доступа, что и члены самого класса. Дружественными могут быть объявлены как классы, так и отдельные функции.

Закрытый член класса (private member). Члены, определенные после спецификатора доступа private; доступный только для друзей и других членов класса. Закрытыми обычно объявляют переменные-члены и вспомогательные функции, используемые классом, но не являющиеся частью интерфейса типа.

Изменяемая переменная-член (mutable data member). Переменная-член, которая никогда не становится константой, даже когда является членом константного объекта. Значение изменяемой переменной-члена вполне может быть изменено в константной функции.

Инкапсуляция (encapsulation). Разделение реализации и интерфейса. Инкапсуляция скрывает детали реализации типа. В языке C++ инкапсуляция предотвращает доступ обычного пользователя класса к его закрытым членам.

Интерфейс (interface). Открытые (public) операции, поддерживаемые типом. Обычно интерфейс не включает переменные-члены.

Класс (class). Механизм языка C++, позволяющий создавать собственные абстрактные типы данных. Классы могут содержать как данные, так и функции. Класс определяет новый тип и новую область видимости.

Ключевое слово class. Следующие после ключевого слова class объявления класса считаются по умолчанию закрытыми (private).

Ключевое слово struct. Следующие после ключевого слова struct объявления структуры считаются по умолчанию открытыми (public).

Константная функция-член (const member function). Функция-член, которая не может изменять обычные (т.е. нестатические и неизменяемые) переменные-члены объекта. Указатель `this` константного члена класса является указателем на константу. Функция-член может быть перегружена на основании того, является ли она константной или нет.

Конструктор (constructor). Специальная функция-член, обычно инициализирующая объекты. Конструктор должен присвоить каждой переменной-члену хорошо продуманное исходное значение.

Конструктор преобразования (converting constructor). Неявный конструктор, который может быть вызван с одиночным аргументом. Конструктор преобразования используется для неявного преобразования типа аргумента в тип класса.

Спецификатор доступа (access specifier). Ключевые слова `public` и `private` определяют, доступны ли данные члены для пользователей класса или только его друзьям и членам. Спецификаторы могут присутствовать многократно в пределах класса. Каждый спецификатор устанавливает степень доступа для последующих членов до следующего спецификатора.

Незавершенный тип (incomplete type). Тип, который уже объявлен, но еще не определен. Использовать незавершенный тип для определения члена класса или переменной нельзя. Однако ссылки или указатели на незавершенные типы вполне допустимы.

Область видимости класса (class scope). Каждый класс определяет область видимости. Область видимости класса сложнее, чем другие области видимости, поскольку определенные в теле класса функции-члены могут использовать имена, которые появятся уже после определения.

Объявление класса (class declaration). Ключевое слово `class` (или `struct`), сопровождаемое именем класса и точкой с запятой. Если класс объявлен, но не определен, то это *незавершенный тип*.

Открытый член класса (public member). Члены, определенные после спецификатора доступа `public`; доступны для любого пользователя класса. Обычно в разделах `public` определяют только те функции, которые определяют интерфейс класса.

Поиск имени (name lookup). Процесс поиска объявления используемого имени.

Предварительное объявление (forward declaration). Объявление имени еще не определенного класса. Как правило, используется для ссылки на объявление класса до его определения. См. *незавершенный тип*.

Реализация (implementation). Как правило, закрытые (`private`) члены класса, определяющие данные и все операции, которые не предназначены для использования кодом, применяющим тип.

Синтезируемый стандартный конструктор (synthesized default constructor). Компилятор самостоятельно создает (синтезирует) стандартный конструктор для классов, у которых не определено никаких конструкторов. Этот конструктор инициализирует переменные-члены типа класса, запуская их стандартные конструкторы, а переменные-члены встроенных типов остаются неинициализированными.

Список инициализации конструктора (constructor initializer list). Перечень исходных значений переменных-членов класса. Инициализация переменных-членов класса значениями списка осуществляется прежде, чем выполняется тело конструктора. Пе-

ременные-члены класса, которые не указаны в списке инициализации, инициализируются неявно, своими значениями по умолчанию.

Стандартный конструктор (default constructor). Конструктор без параметров.

Указатель `this`. Значение, неявно передаваемое как дополнительный аргумент каждой нестатической функции-члену. Указатель `this` указывает на объект, функция которого вызывается.

Функция-член (member function). Член класса, являющийся функцией. Обычные функции-члены связаны с объектом класса при помощи неявного указателя `this`. Статические функции-члены с объектом не связаны и указателя `this` не имеют. Функции-члены вполне могут быть перегружены; если это так, то неявный указатель `this` участвует в подборе функции.

Явный конструктор (explicit constructor). Конструктор с одним аргументом, который, однако, не может быть использован для неявного преобразования. Объявление явного конструктора предваряется ключевым словом `explicit`.

